

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский
Нижегородский государственный университет им. Н.И. Лобачевского»
(ННГУ)

Институт информационных технологий, математики и механики

**Кафедра: высокопроизводительных вычислений и системного
программирования**

Направление подготовки: «Программная инженерия»
Магистерская программа: «Технологии цифровой трансформации»

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

Тема:

**«Разработка пользовательского интерфейса для университетской
образовательной платформы»**

Выполнил:

студент группы 3824М1ПР1

_____ Юнин Д.Д.

Подпись

Научный руководитель:

доцент кафедры ВВСП, к.т.н.,

_____ Сысоев А.В.

Подпись

Нижний Новгород

2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	5
1. ИЗУЧЕНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ И АНАЛИЗ ТРЕБОВАНИЙ К LMS-ПЛАТФОРМАМ.....	7
1.1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ: СТРУКТУРА И УЧАСТНИКИ УЧЕБНОГО ПРОЦЕССА В ВЫСШЕМ УЧЕБНОМ ЗАВЕДЕНИИ	7
1.1.1. Терминологический глоссарий предметной области.....	7
1.1.2. Организационно-структурная модель и её отображение в SPA-архитектуре.....	8
1.1.3. Жизненный цикл и состояния ключевых сущностей.....	9
1.2. ОБЗОР И СРАВНИТЕЛЬНЫЙ АНАЛИЗ СУЩЕСТВУЮЩИХ СИСТЕМ УПРАВЛЕНИЯ ОБУЧЕНИЕМ (LMS)	10
1.3. РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ К КЛИЕНТСКОЙ ЧАСТИ ПЛАТФОРМЫ.....	13
1.3.1. Общие функциональные требования.....	13
1.3.2. Функциональные требования к интерфейсу Администратора.....	13
1.3.3. Функциональные требования к интерфейсу Преподавателя.....	14
1.3.4. Функциональные требования к интерфейсу Студента	15
1.4. РАЗРАБОТКА НЕФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ	15
1.4.1. Требования к интерфейсу пользователя.....	15
1.4.2. Производительность и оптимизация.....	16
1.4.3. Безопасность и разграничение доступа	16
1.4.4. Надежность и отказоустойчивость.....	17
1.5. ПРОЕКТИРОВАНИЕ И ОПИСАНИЕ ПОЛЬЗОВАТЕЛЬСКИХ СЦЕНАРИЕВ.....	17
1.5.1. Сценарии взаимодействия для роли «Администратор».....	17
1.5.2. Сценарии взаимодействия для роли «Преподаватель».....	19
1.5.3. Сценарии взаимодействия для роли «Студент»	22
2. АРХИТЕКТУРНЫЙ ПОДХОД К ПРОЕКТИРОВАНИЮ КЛИЕНТСКОЙ ЧАСТИ	
ПРИЛОЖЕНИЯ	24
2.1. ПРОБЛЕМЫ ТРАДИЦИОННОГО ПОДХОДА К ОРГАНИЗАЦИИ КОДОВОЙ БАЗЫ ВО ФРОНТЕНД-РАЗРАБОТКЕ	24
2.2. МЕТОДОЛОГИЯ FEATURE-SLICED DESIGN: ОСНОВНЫЕ ПОНЯТИЯ И СТРУКТУРА СЛОЕВ.....	26
2.2.1. Слои.....	27
2.2.2. Слайсы.....	29
2.2.3. Сегменты.....	29
2.3. КЛЮЧЕВЫЕ ПРИНЦИПЫ FSD: ПУБЛИЧНЫЙ API, НИЗКАЯ СВЯЗАННОСТЬ И ВЫСОКОЕ ЗАЦЕПЛЕНИЕ.....	30

2.3.1. Принцип публичного API	30
2.3.2. Низкая связность	31
2.3.3. Высокое зацепление	31
2.4. СРАВНИТЕЛЬНЫЙ АНАЛИЗ FSD С ДРУГИМИ АРХИТЕКТУРАМИ ФРОНТЕНД-ПРИЛОЖЕНИЙ	33
3. ИНСТРУМЕНТАЛЬНЫЕ СРЕДСТВА РАЗРАБОТКИ	36
3.1. СБОРКА	36
3.1.1. Vite	36
3.2. ЯДРО ПРИЛОЖЕНИЯ	37
3.2.1. Vue 3 (composition API)	37
3.2.2. TypeScript	38
3.3. СОСТОЯНИЕ И НАВИГАЦИЯ	39
3.3.1. Pinia	39
3.3.2. Vue router	40
3.4. ИНТЕРФЕЙС	41
3.4.1. PrimeVue	41
3.4.2. Vue i18n	41
3.5. ДАННЫЕ И ФОРМЫ	42
3.5.1. TanStack Query (Vue query)	42
3.5.2. Vee-validate и zod	43
3.6. КАЧЕСТВО И РАЗРАБОТКА	43
3.6.1. Vitest	43
3.6.2. Storybook	44
3.6.3. ESLint и Prettier	44
3.6.4. Husky	45
3.7. ОКРУЖЕНИЕ И РАЗВЕРТЫВАНИЕ	45
3.7.1. Json-server	45
3.7.2. Docker и nginx	47
3.8. ОПИСАНИЕ РЕЗУЛЬТАТОВ	47
4. РАЗРАБОТКА КЛИЕНТСКОГО ПРОГРАММНОГО МОДУЛЯ ПЛАТФОРМЫ НА ОСНОВЕ МЕТОДОЛОГИИ FSD	51
4.1. АРХИТЕКТУРНАЯ СТРУКТУРА ПРОЕКТА И ДЕКОМПОЗИЦИЯ ПО СЛОЯМ FSD	51
4.1.1. Слой shared	53
4.1.2. Слой entities	53
4.1.3. Слой features	54
4.1.4. Слой widgets	55
4.1.5. Слой pages	55
4.1.6. Слой app	56
4.2. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ КЛЮЧЕВЫХ СЦЕНАРИЕВ	57

4.2.1. Реализация системы аутентификации, авторизации и динамического ролевого разграничения доступа.....	57
4.2.2. Клиентские компоненты интерфейса администратора.....	62
4.2.3. Функциональный модуль преподавателя.....	72
4.2.4. Интерактивный интерфейс студента	81
5. ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ И АВТОМАТИЗАЦИЯ ПРОЦЕССОВ РАЗРАБОТКИ.....	86
5.1. ОРГАНИЗАЦИЯ АВТОМАТИЧЕСКИХ ПРОВЕРОК КАЧЕСТВА КОДА С ИСПОЛЬЗОВАНИЕМ GIT-ХУКОВ.....	86
5.1.1. Назначение и место в процессе разработки	86
5.1.2. Git-хуки, Husky и lint-staged.....	87
5.1.3. Описание результатов	87
5.2. МОДУЛЬНОЕ И ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ БИЗНЕС-ЛОГИКИ С ИСПОЛЬЗОВАНИЕМ VITEST.....	89
5.2.1. Назначение и выбор инструмента.....	89
5.2.2. Конфигурация и структура тестов	89
5.2.3. Тесты чистых функций и хелперов.....	90
5.2.4. Интеграционные тесты.....	91
5.2.5. Unit тесты функций форм.....	91
5.2.6. Компонентные тесты	92
5.2.7. Покрытие кода.....	94
5.3. ДОКУМЕНТИРОВАНИЕ И ИЗОЛИРОВАННАЯ РАЗРАБОТКА КОМПОНЕНТОВ В СРЕДЕ STORYBOOK.....	95
5.3.1. Назначение и концепция	95
5.3.2. Конфигурация и среда выполнения	95
5.3.3. Структура stories в FSD	96
5.3.4. Interaction tests и проверка доступности.....	100
ЗАКЛЮЧЕНИЕ	101
СПИСОК ЛИТЕРАТУРЫ	103

Введение

Одной из фундаментальных задач высшего образования является не только передача теоретических знаний, но и формирование у студентов практических навыков. Для специалиста в области программной инженерии особую ценность представляет опыт прохождения полного жизненного цикла разработки программного обеспечения: от анализа предметной области и формализации требований - до проектирования архитектуры, реализации функциональных модулей и организации процессов обеспечения качества.

Современная индустрия разработки программного обеспечения характеризуется стремительным развитием технологий и инструментов. Фронтенд-разработка переживает период активной трансформации: на смену традиционному серверному рендерингу приходят высокопроизводительные одностраничные приложения (SPA) [3,4], появляются зрелые методологии организации кодовой базы, развиваются экосистемы современных JavaScript-фреймворков. Для будущего специалиста крайне важно не только знать о существовании этих технологий, но и иметь практический опыт их применения в условиях реального проекта достаточной сложности.

Именно этим обусловлен выбор темы настоящей работы. Предметом исследования и разработки является клиентская часть образовательной платформы - веб-приложение, предоставляющее единое цифровое пространство для трех категорий участников учебного процесса: администраторов, преподавателей и студентов. Необходимо подчеркнуть, что создание полноценной конкурентной альтернативы зрелым коммерческим или академическим LMS-системам [1,2], таким как Moodle, Canvas или Open edX, не является целью данной работы. Основная научно-практическая ценность состоит в системном изучении и применении современных подходов к разработке клиентских веб-приложений на примере предметной области, обладающей достаточной сложностью и разнообразием решаемых задач.

В качестве технологической основы выбран Vue 3 с Composition API подходом [11] в сочетании с методологией Feature-Sliced Design (FSD) [27] - современным стандартом архитектурной организации фронтенд-приложений. Сочетание реактивного фреймворка, строгой архитектурной методологии и инструментов обеспечения качества кода позволяет продемонстрировать подход к разработке, при котором масштабируемость и поддерживаемость системы закладываются на уровне архитектурных решений.

Работа охватывает следующие направления исследования и разработки. В первой главе проводится анализ предметной области, включающий формализацию структуры и участников учебного процесса, сравнительный обзор существующих LMS-решений и разработку функциональных и нефункциональных требований к системе. Вторая глава посвящена архитектурному проектированию: в ней обосновывается выбор методологии Feature-Sliced Design и анализируются ее преимущества перед традиционными подходами к организации кодовой базы. В третьей главе описываются инструментальные средства разработки, обосновывается выбор технологического стека и рассматривается каждый инструмент с точки зрения его роли в проекте. Четвертая глава представляет практическую реализацию: описывается декомпозиция проекта по слоям FSD и приводится детальное описание разработанных программных модулей для каждого пользовательского портала. В пятой, заключительной, главе рассматривается организация процессов обеспечения качества: статический анализ кода, модульное и компонентное тестирование, а также документирование интерфейса в среде Storybook

1. Изучение предметной области и анализ требований к LMS-платформам

1.1. Анализ предметной области: структура и участники учебного процесса в высшем учебном заведении

1.1.1. Терминологический глоссарий предметной области

В рамках автоматизируемой образовательной среды ключевые сущности учебного процесса формализуются следующим образом.

Пользователь (User) - субъект системы, обладающий уникальными учётными данными для входа и назначенной ролью, которая определяет набор доступных интерфейсов и операций. Система поддерживает три основные роли: Администратор, Преподаватель и Студент.

Роль (Role) - именованный набор разрешений, закреплённый за пользователем. На уровне клиентского приложения роль управляет видимостью компонентов навигации, доступностью маршрутов и составом отображаемых элементов управления. Проверка роли осуществляется на клиенте на основе декодирования JWT-токена [30], полученного при аутентификации.

Электронный учебный курс (Course) - структурированная единица образовательного контента, выступающая в роли контейнера для учебных материалов, метаданных и пользователей. На уровне клиентского приложения курс представляется в виде иерархического дерева тем и элементов с возможностью перехода между разделами без перезагрузки страницы.

Элемент курса (Course Element) - минимальная неделимая единица учебного контента. Сущность полиморфна: на клиенте реализуются типы «видеоматериал», «PDF-документ», «презентация» и «тест», каждый из которых требует собственного компонента отображения.

Банк вопросов (Question Bank) - изолированное логическое хранилище тестовых и контрольных заданий, категоризованных по темам, уровням сложности и типам ответов. Используется преподавателем для статической или динамической сборки тестов. Архитектурная изолированность банка позволяет переиспользовать вопросы в нескольких тестах без дублирования.

Тест (Assessment) - контрольное мероприятие, собранное из вопросов банка. Тест обладает набором параметров прохождения: ограничение по времени, срок сдачи, проходной балл. На клиенте тест порождает сессию прохождения.

Сессия тестирования (Assessment Session) - временное состояние, возникающее в момент начала прохождения теста студентом. Сессия хранит: идентификатор студента, снимок вопросов, текущие ответы, оставшееся время и статус завершения.

Оценка (Grade) - количественный результат прохождения теста или выполнения задания. Для тестов с выбором ответа оценка вычисляется автоматически сервером; для заданий со свободным ответом требует ручной проверки преподавателем.

Балльно-рейтинговая система (БРС) - накопительная модель оценки успеваемости, агрегирующая оценки за все элементы курса в итоговый рейтинг студента. Требует от клиентского интерфейса динамических аналитических дашбордов и интерактивных ведомостей для преподавателей.

Академическая группа (Academic Group) - совокупность пользователей с ролью «Студент», объединённых общим годом поступления и направлением подготовки. Закрепление группы за курсом определяет, для каких студентов курс будет отображаться в их личных кабинетах.

1.1.2. Организационно-структурная модель и её отображение в SPA-архитектуре

Организационная структура университета имеет строго иерархический характер. С точки зрения информационного моделирования вложенность сущностей вуза описывается следующей цепочкой:

Университет → Институт / Факультет → Кафедра → Направление подготовки → Академическая группа → Студент

Данная иерархия напрямую проецируется на архитектуру Single Page Application, определяя структуру клиентского роутинга и интерфейсные паттерны.

Динамический клиентский роутинг. Иерархия вуза задаёт вложенную структуру динамических маршрутов в менеджере маршрутов. Каждый уровень иерархии соответствует сегменту URL с параметром-идентификатором, что позволяет компонентам страниц извлекать нужный идентификатор из параметров маршрута и выполнять соответствующий API-запрос без передачи данных через глобальное хранилище.

Древовидные компоненты интерфейса. Для визуализации иерархии в административной панели применяются интерактивные древовидные компоненты. При помощи их администратору становится проще воспринимать связи между кафедрами, направлениями и группами, а также выполнять операции добавления или удаления узлов в рамках одной страницы.

Каскадный выбор. Для создания учётных записей пользователей или назначения преподавателям определённых структурных единиц используются каскадные выпадающие списки. Клиентское приложение динамически фильтрует список дочерних элементов при выборе родительского: выбор института ограничивает список направлений, выбор направлений - список групп.

Изолированные порталы по ролям. Каждая пользовательская роль обслуживается собственным набором защищённых маршрутов. Попытка перехода к маршруту, который принадлежит другой роли, вызывает автоматическое перенаправление.

1.1.3. Жизненный цикл и состояния ключевых сущностей

Для обеспечения корректного поведения интерфейса, состояния ключевых сущностей описываются в виде конечных автоматов: в зависимости от текущего состояния меняется логика отображения данных.

Состояния курса:

Черновик (Draft) - начальное состояние при создании. Курс доступен для редактирования преподавателю, но полностью скрыт от студентов.

Активный (Active) - курс опубликован. Отображается на дашборде студентов прикрепленных групп, открывается доступ к материалам и тестам.

Архивный (Archived) - курс завершён. Для студентов переходит в режим «только чтение», убирается с главного дашборда и перемещается в раздел истории.

Состояния задания (практической работы):

Не начато (Not Started) - задание доступно, кнопка отправки активна.

В процессе (In Progress) - студент сохранил черновик ответа.

На проверке (Pending Review) - решение отправлено. У студента блокируется повторная отправка; у преподавателя запись подсвечивается как требующая проверки.

Проверено (Graded) - преподаватель выставил оценку и оставил комментарий. Редактирование заблокировано для обеих сторон.

Состояния тестовой сессии:

Доступен (Available) - тест опубликован, кнопка запуска активна.

Выполняется (In Session) - студент запустил тест. Активирован таймер обратного отсчета;

Завершён (Submitted) - время вышло или студент нажал «Завершить». Данные отправлены на сервер, запускается автоматическая проверка.

Оценен (Graded) - результаты доступны студенту.

1.2. Обзор и сравнительный анализ существующих систем управления обучением (LMS)

Для обоснованного выбора архитектурных решений и интерфейсных паттернов необходимо провести анализ существующих систем управления обучением [1,2]. Среди платформ, широко применяемых в академической среде, наибольший интерес представляют три системы: Moodle, Canvas LMS и Stepik. Они охватывают спектр от монолитных решений с открытым исходным кодом до современных высоконагруженных облачных SaaS-платформ.

Moodle

Moodle (Modular Object-Oriented Dynamic Learning Environment) - наиболее распространённая платформа в российских вузах [1]. Это монолитная система с открытым исходным кодом, написанная на PHP. Главное достоинство Moodle - развитая экосистема плагинов и глубокая проработка академических сценариев: ведение журналов, учёт групп и иерархических структур вуза, интеграция с деканатом.

Однако с точки зрения фронтенд-архитектуры Moodle имеет существенные недостатки. Система использует классический серверный рендеринг на основе шаблонизатора Mustache и устаревших JS-библиотек (jQuery, RequireJS). Это приводит к полной перезагрузке страницы при каждом переходе, медленному отклику интерфейса и перегруженному DOM-дереву. Реализация плавных анимаций и интерактивного тестирования в реальном времени в рамках такой архитектуры сопряжена со значительными трудозатратами.

Canvas LMS

Canvas LMS от компании Instructure построен на базе Ruby on Rails и использует React в качестве фронтенд-стека. Платформа предлагает более отзывчивый и визуально чистый

интерфейс по сравнению с Moodle: часть экранов реализована как React SPA внутри монолитного приложения.

Тем не менее клиентская часть Canvas глубоко интегрирована в серверный монолит, что усложняет кастомизацию. Самостоятельное развёртывание Canvas LMS представляет собой сложную инженерную задачу из-за жёсткой привязки к облачной инфраструктуре AWS. Платформа ориентирована на западный SaaS-рынок и слабо поддерживает особенности российской академической среды.

Stepik

Stepik - современная российская облачная платформа, ориентированная на интерактивные курсы и автоматическую проверку задач. Клиентская часть реализована как высокопроизводительное SPA на базе React. Stepik демонстрирует отличный пользовательский опыт: мгновенные переходы между страницами, плавно работающий таймер тестов, удобный конструктор шагов курса.

Принципиальное ограничение - исключительно SaaS-модель без возможности развёртывания на серверах университета, что исключает интеграцию с внутренними базами данных вуза и соответствие требованиям законодательства о персональных данных. Кроме того, в Stepik отсутствуют инструменты администрирования сложной вузовской структуры (кафедры, направления, закреплённые академические группы).

Сравнительная таблица

Таблица 1.

Результаты анализа архитектур и возможностей существующих LMS

Критерий сравнения	Moodle	Canvas LMS	Stepik
Архитектура клиента	SSR (PHP + Mustache)	Гибридная (Rails + React)	SPA (React)
Скорость навигации	Низкая (полная перезагрузка)	Средняя	Высокая (без перезагрузки)
Адаптивность (mobile)	Удовлетворительная	Хорошая	Отличная

Возможность on-premise	Полная	Затруднённая	Отсутствует
Поддержка иерархии вуза	Полная	Частичная	Ограниченная
Ролевая модель	Развитая	Средняя	Упрощённая
Тип лицензии	Open Source (GPL)	Open Source / SaaS	SaaS (закрытый)
Автосохранение сессии теста	Частичное	Есть	Есть

Выводы и заимствованные практики

Проведенный анализ [1,2] позволяет выделить четыре направления лучших практик, которые служат ориентирами при проектировании разрабатываемой платформы.

SPA-архитектура клиента. Все сценарии взаимодействия, требующие высокой отзывчивости - прохождение теста, редактирование содержания курса, навигация по структуре - реализуются на клиенте без полной перезагрузки страницы. Vue Router обеспечивает мгновенные переходы между экранами внутри одного SPA.

Глубокая ролевая модель и поддержка иерархии вуза. Система учитывает реальную структуру российского вуза: университет → институт → кафедра → направление → группа → студент. Закрепление пользователей за структурными подразделениями определяет область видимости контента в интерфейсе каждой роли.

Изолированные компоненты для типов контента. Каждый тип элемента курса (видео, PDF, тест, задание) реализуется как самостоятельный полиморфный компонент с собственной логикой отображения, не зависящей от соседних типов.

Надёжная сессия тестирования с автосохранением. Промежуточные ответы студента автоматически сохраняются на сервер по мере заполнения. При случайном закрытии вкладки или сетевом сбое сессия восстанавливается с последнего сохранённого состояния.

При этом цель данной работы не состоит в создании прямой альтернативы перечисленным коммерческим и академическим решениям. Основная научно-практическая задача заключается в

прохождении полного жизненного цикла разработки сложной образовательной веб-платформы с применением современного технологического стека и методологий.

1.3. Разработка функциональных требований к клиентской части платформы

Функциональные требования определяют то, какие возможности должна предоставлять система пользователям. В ходе анализа были формализованы и систематизированы функциональные требования для трех ключевых ролей пользователей: Администратора, Преподавателя и Студента.

Ниже приведено описание функциональных требований к платформе к интерфейсу каждого участника образовательного процесса.

1.3.1. Общие функциональные требования

Перед началом работы с платформой клиентская часть должна обеспечивать прохождение сквозных процедур, обязательных для всех категорий пользователей:

Идентификация и аутентификация: каждый пользователь системы должен проходить процедуру авторизации с помощью пары уникального логина и пароля.

Разграничение прав доступа: после успешного входа система определяет назначенную роль пользователя и на клиентском уровне динамически формирует доступный набор интерфейсов, меню навигации и защищенных маршрутов.

Сброс и восстановление пароля: клиентская часть должна содержать сценарий самостоятельного восстановления пароля пользователем через указание электронной почты, связанной с учетной записью, и последующее подтверждение изменений.

1.3.2. Функциональные требования к интерфейсу Администратора

Интерфейс Администратора ориентирован на техническое обеспечение работоспособности системы, управление иерархической структурой вуза и ведение базы данных пользователей:

Управление учетными записями пользователей: Администратор должен иметь возможность создавать новые учетные записи (как вручную для единичных пользователей, так и пакетно посредством импорта файлов в формате Excel), редактировать существующие профили (изменять электронную почту, переназначать группы) и удалять неактуальные аккаунты.

Управление иерархической структурой университета: интерфейс должен предоставлять инструменты для визуального редактирования древовидной структуры направлений подготовки, институтов/факультетов, кафедр и академических групп.

Управление правами доступа преподавателей: Администратор закрепляет за конкретным преподавателем список доступных ему учебных направлений, институтов или групп, определяя его зону ответственности в системе.

1.3.3. Функциональные требования к интерфейсу Преподавателя

Интерфейс Преподавателя предназначен для создания учебного контента, контроля успеваемости студентов и управления учебными курсами:

Управление курсами: возможность создания новых курсов с указанием названия, описания и закрепленных академических групп студентов. Преподаватель может переводить курсы в архивный статус и добавлять других преподавателей в качестве соавторов.

Конструирование содержания: интерфейс позволяет выстраивать структуру курса и наполнять ее учебными материалами разных форматов (текст, презентации, PDF-документы, видеофайлы).

Управление банком вопросов: создание и ведение централизованной базы вопросов различных типов (тестовые с одиночным или множественным выбором, контрольные со свободным кратким или развернутым ответом). Вопросам могут назначаться параметры сложности.

Создание тестов и контрольных заданий: сборка тестов из готовых банков вопросов, установка проходного балла, ограничения по времени на прохождение и жестких сроков сдачи.

Мониторинг успеваемости студентов: Преподавателю предоставляется детальная статистика по курсу в виде интерактивных графиков и таблиц, отражающих прогресс прохождения тем, результаты тестирования и общую успеваемость групп. Должен быть реализован экспорт этих отчетов в файлы формата Excel.

Проверка работ: интерфейс должен содержать удобный инструмент для ручной проверки развернутых ответов студентов на контрольные задания с возможностью выставления оценки и отправки текстового комментария.

1.3.4. Функциональные требования к интерфейсу Студента

Интерфейс Студента представляет собой адаптивное рабочее пространство, спроектированное для максимально комфортного и сфокусированного прохождения обучения:

Дашборд доступных курсов: Студент должен видеть наглядный перечень всех учебных курсов, на которые он записан.

Взаимодействие с учебным материалом: навигация по структуре курса, последовательное чтение лекций, просмотр прикрепленных видео, а также предпросмотр и скачивание методических материалов (PDF, презентаций).

Прохождение тестовых работ: интерактивное выполнение тестов (с визуальным отображением оставшегося времени) и отправка решений практических или контрольных работ (включая загрузку файлов с выполненными заданиями).

Мониторинг успеваемости: доступ к разделу с собственными оценками по всем элементам курса, обратной связью от преподавателей и общим индикатором прогресса прохождения учебной программы.

1.4. Разработка нефункциональных требований

Нефункциональные требования определяют качественные характеристики клиентской части платформы, накладывая ограничения на архитектурные решения, производительность, безопасность и пользовательский опыт. Специфика разработки клиентского SPA-приложения для академической среды обуславливает следующие нефункциональные требования:

1.4.1. Требования к интерфейсу пользователя

Интуитивность и доступность: интерфейс должен быть интуитивно понятным для пользователей с разным уровнем компьютерной грамотности. Навигация должна строиться по принципу минимального количества кликов для достижения целевого действия.

Кроссплатформенная адаптивность: клиентская часть должна корректно отображаться и сохранять полную функциональность на широком спектре устройств: персональные компьютеры, ноутбуки, планшеты.

Доступность: обязательна поддержка стандартов доступности для людей с ограниченными возможностями здоровья (контрастные цветовые схемы, масштабирование текста без разрушения верстки, поддержка навигации с клавиатуры).

1.4.2. Производительность и оптимизация

Время первичной загрузки: размер исходного бандла приложения должен быть минимизирован за счет разделения кода и ленивой загрузки некритичных модулей. Время до первой отрисовки контента (FCP) при стандартном интернет-соединении должно составлять не более 1.5–2 секунд.

Отзывчивость интерфейса: переходы между страницами внутри Single Page Application должны происходить плавно, без визуального мерцания и полной перезагрузки страниц. Для сглаживания времени ожидания ответов от сервера необходимо использовать анимированные скелетоны.

Управление состоянием: хранение глобального состояния приложения должно исключать лишние повторные сетевые запросы при переключении между вкладками или возврате на предыдущие страницы.

1.4.3. Безопасность и разграничение доступа

Защита каналов связи: весь обмен данными между клиентской и серверной частями платформы должен осуществляться по защищенному протоколу HTTPS с использованием современных алгоритмов шифрования.

Безопасное хранение сессий: авторизационные токены (JWT) [30] должны храниться в защищенных клиентских хранилищах, минимизирующих риски XSS-атак (рекомендуется использование HttpOnly Cookie или защищенного стейта приложения).

Строгая изоляция данных: на клиентском уровне должна быть жестко заблокирована возможность просмотра данных, не принадлежащих текущему пользователю. Студент может видеть только свои оценки, а Преподаватель - только данные студентов, привязанных к его

курсам. Изменение персональных данных пользователей и иерархии вуза должно быть недоступно в интерфейсе Преподавателя и Студента.

1.4.4. Надежность и отказоустойчивость

Обработка сетевых ошибок: в случае кратковременного обрыва связи или недоступности API бэкенда клиентская часть должна корректно обрабатывать эти ситуации (выводить понятные пользователю предупреждения с предложением повторить попытку), предотвращая зависание интерфейса или потерю введенных данных, например, ответов на вопросы теста.

Автоматическое сохранение прогресса: при прохождении студентом тестирования его промежуточные ответы должны автоматически кэшироваться или отправляться на сервер, чтобы в случае случайного закрытия вкладки или сбоя компьютера студент мог продолжить выполнение задания с того же места.

1.5. Проектирование и описание пользовательских сценариев

Для детальной спецификации функциональных требований и последующего проектирования интерфейсов необходимо формализовать основные сценарии взаимодействия пользователей с платформой. Каждый сценарий описывает последовательность действий актора и реакцию системы, определяя прецеденты использования клиентской части приложения.

1.5.1. Сценарии взаимодействия для роли «Администратор»

1.5.1.1. Ручное создание учетной записи пользователя

Предусловия: Администратор авторизован в системе и находится в административной панели.

Основной сценарий (Шаги):

1. Администратор переходит на страницу «Управление пользователями».
2. Нажимает кнопку «Создать пользователя».
3. Заполняет интерактивную форму ввода данных (ФИО, уникальный логин, email, выбор роли: Администратор, Преподаватель, Студент).

4. Нажимает кнопку «Сохранить».

Результат: Данные отправляются на сервер, в базе данных создается запись, а клиентская часть отображает уведомление об успешном создании.

1.5.1.2. Импорт учетных записей из внешнего файла

Предусловия: Администратор подготовил файл со списком пользователей в формате Excel.

Основной сценарий (Шаги):

1. Администратор переходит в раздел «Управление пользователями».
2. Нажимает кнопку «Импортировать из файла».
3. В диалоговом окне выбирает подготовленный файл и нажимает «Загрузить».
4. Администратор подтверждает действие нажатием кнопки «Импортировать».

Результат: Пакет данных отправляется на сервер, в системе регистрируются новые пользователи, интерфейс выводит отчет об успешном импорте и количестве созданных записей.

1.5.1.3. Модификация организационной структуры вуза

Предусловия: Администратор находится в разделе «Иерархия структуры».

Основной сценарий (Шаги):

1. Администратор просматривает древовидную структуру направлений подготовки, институтов и групп.
2. Выбирает необходимый узел дерева (например, конкретный Институт) и нажимает кнопку «Добавить дочерний элемент» или «Редактировать».
3. Вводит новое название (например, название создаваемой академической группы) и сохраняет изменения.

Результат: Дерево структуры динамически перерисовывается на клиенте, отправляя обновленную конфигурацию иерархии на сервер.

1.5.1.4. Просмотр и редактирование профиля пользователя

Предусловия: Пользователь зарегистрирован в системе.

Шаги:

1. Администратор переходит на страницу «Пользователи».
2. Использует фильтры и строку поиска для нахождения нужного человека.
3. Кликает на имя пользователя для открытия карточки профиля.
4. Вносит необходимые изменения (корректирует email, переназначает учебную группу или направление).
5. Нажимает кнопку «Сохранить изменения».

Результат: Обновленная информация отправляется на сервер, локальный стек данных на клиенте обновляется, выводится уведомление об успешном сохранении.

1.5.2. Сценарии взаимодействия для роли «Преподаватель»

1.5.2.1. Создание учебного курса

Предусловия: Преподаватель авторизован в системе.

Шаги:

1. На главной странице нажимает кнопку «Добавить курс».
2. Вводит название и подробное описание курса.
3. При необходимости указывает соавторов курса из числа других преподавателей.
4. Нажимает кнопку «Сохранить изменения».

Результат: Курс создается и становится доступным для редактирования и наполнения в личном кабинете преподавателя.

1.5.2.2. Структурирование содержания курса

Предусловия: Учебный курс успешно создан.

Шаги:

1. Преподаватель переходит в раздел «Содержание курса».
2. Нажимает кнопку добавления новой темы (например, «Раздел 1»).
3. Внутри созданной темы нажимает «Добавить подтему».
4. Нажимает кнопку «Сохранить изменения».

Результат: На клиенте формируется иерархическая структура курса, готовая к наполнению материалами.

1.5.2.3. Добавление учебных материалов в курс

Предусловия: Структура курса (разделы и подразделы) сформирована.

Шаги:

1. Преподаватель выбирает нужную подтему в структуре курса.
2. Нажимает кнопку «Добавить материал».
3. Выбирает тип материала (видео, презентация, PDF-документ, текстовый файл) и загружает его в систему.
4. Указывает название материала и краткое описание, после чего нажимает «Сохранить».

Результат: Материал прикрепляется к подтеме курса и становится доступен для изучения студентам.

1.5.2.4. Добавление вопросов в банк вопросов

Предусловия: Преподаватель находится в разделе «Банк вопросов».

Шаги:

1. Выбирает существующий банк из списка или создает новый нажатием на «Добавить банк вопросов».
2. Внутри выбранного банка нажимает кнопку «Добавить вопрос».
3. Определяет тип вопроса (тестовый с вариантами, контрольный с текстовым вводом).
4. Заполняет текст вопроса и указывает сложность.
5. Для тестового вопроса добавляет варианты ответов и отмечает правильные; для программного - указывает критерии оценки и тесты.
6. Нажимает «Сохранить вопрос».

Результат: Вопрос заносится в базу данных банка вопросов и готов к использованию в тестах.

1.5.2.5. Создание теста на основе банка вопросов

Предусловия: Создана подтема курса, сформирован и наполнен банк вопросов.

Шаги:

1. Преподаватель переходит в нужную подтему курса и нажимает «Добавить тест».
2. Вводит название теста, описание и параметры прохождения (время выполнения, дедлайны).
3. Переходит во вкладку «Вопросы» и нажимает «Выбрать вопросы из банка».
4. Фильтрует вопросы по типу и сложности, отмечает галочками нужные и импортирует их.
5. Настраивает количество начисляемых баллов за каждый верный ответ.
6. Нажимает «Сохранить тест».

Результат: Тест публикуется в структуре курса и становится доступен для прохождения студентам.

1.5.2.6. Проверка контрольных заданий студентов

Предусловия: Студенты выполнили контрольные работы, требующие ручной оценки.

Шаги:

1. Преподаватель переходит в раздел статистики и успеваемости курса.
2. Выбирает необходимое задание из списка требующих проверки.
3. Просматривает прикрепленные файлы решений студентов.
4. Выставляет оценку в баллах (вручную) и при необходимости пишет комментарий с разбором ошибок.
5. Нажимает «Сохранить оценки».

Результат: Оценки фиксируются в системе, студент получает уведомление, а общая ведомость успеваемости автоматически пересчитывается.

1.5.2.7. Отслеживание успеваемости и экспорт отчетов

Предусловия: Студенты начали прохождение курса.

Шаги:

1. Преподаватель переходит в раздел статистики курса.
2. Выбирает академическую группу или конкретного студента для анализа.
3. Изучает сводный прогресс прохождения тем и результаты сданных тестов.
4. Нажимает кнопку «Экспорт».
5. Выбирает желаемый формат файла (Excel) и скачивает его на устройство.

Результат: На устройство скачивается файл с актуальными данными об успеваемости студентов.

1.5.2.8. Назначение учебных групп на курс

Предусловия: Курс создан, Администратор назначил преподавателя на соответствующие группы.

Шаги:

1. Преподаватель заходит в настройки нужного курса.
2. Переходит во вкладку «Доступ групп».
3. Из предложенного перечня доступных ему групп отмечает галочками те, для которых открывается данный курс.
4. Нажимает кнопку «Сохранить изменения».

Результат: Курс появляется в личных кабинетах всех студентов выбранных академических групп.

1.5.3. Сценарии взаимодействия для роли «Студент»

1.5.3.1. Прохождение разделов учебного курса

Предусловия: Студент авторизован на платформе и записан на курс.

Шаги:

1. Студент открывает список своих курсов и переходит на нужный курс.
2. Система выводит структуру доступных материалов.
3. Студент выбирает лекцию или практический материал для изучения.
4. В зависимости от типа контента: изучает текст на странице, просматривает видеоматериал или скачивает прикрепленный PDF-документ (презентацию).
5. Использует систему навигации для быстрого перехода к следующей теме курса.

Результат: В системе фиксируется факт изучения темы, прогресс прохождения курса студентом увеличивается.

1.5.3.2. Прохождение тестовой работы

Предусловия: Студенту доступен опубликованный в курсе тест.

Шаги:

1. Студент открывает список материалов курса и выбирает активный тест.
2. Ознакомляется с параметрами (количество попыток, время) и нажимает кнопку начала тестирования.
3. Последовательно отвечает на вопросы, выбирая варианты ответов или вписывая краткие ответы с клавиатуры.
4. Контролирует оставшееся время по таймеру на экране.
5. Нажимает кнопку «Завершить тест».

Результат: Система фиксирует ответы, отправляет их на автоматическую проверку и (в зависимости от настроек теста) сразу выводит набранные баллы и детальный отчет по ошибкам.

2. Архитектурный подход к проектированию клиентской части приложения

2.1. Проблемы традиционного подхода к организации кодовой базы во фронтенд-разработке

В течение длительного времени стандартным методом организации проектов во фронтенд-разработке являлась декомпозиция кодовой базы по техническому типу. При таком подходе структура каталогов приложения верхнего уровня выглядит следующим образом:

- **components** - содержит все компоненты интерфейса
- **views** - для компонентов целых страниц;
- **store** - для файлов управления глобальным состоянием приложения;
- **api** - для функций отправки сетевых запросов и описания сетевых моделей;
- **utils** - для вспомогательных функций;
- **hooks** - для переиспользуемой реактивной логики.

На начальных этапах жизненного цикла программного продукта техническая декомпозиция обеспечивает быстрый старт разработки за счет простоты и интуитивности структуры. Однако по мере развития приложения, усложнения бизнес-логики и увеличения команды разработчиков традиционный подход порождает ряд серьезных архитектурных проблем, снижающих качество программного обеспечения и увеличивающих стоимость его поддержки.

Техническая размытость и разрастание папок общего назначения

По мере роста проекта папка `components` неизбежно превращается в плоское неструктурированное хранилище, содержащее сотни файлов. Разработчик сталкивается с ситуацией, когда компонент кнопки находится в той же директории, что и сложный интерактивный модуль. В рамках такой структуры невозможно определить зону ответственности компонента, степень его важности для системы и безопасность внесения изменений. Попытки структурировать данную папку путем создания вложенных директорий, например, `components/common`, `components/modals`, `components/layout`, носят субъективный характер и со временем приводят к хаосу из-за отсутствия жестких архитектурных правил.

Низкое зацепление кода

При декомпозиции код, отвечающий за реализацию одной конкретной бизнес-фичи, оказывается распределен по всему дереву проекта. Например, для добавления или изменения функционала «Прохождение теста студентом» разработчик вынужден одновременно вносить изменения, например, в следующие файлы:

- `/components/TestPlayer.vue` - для разметки интерфейса;
- `/store/test.ts` - для изменения глобального состояния сессии;
- `/api/test.ts` - для добавления новых запросов к бэкенду;
- `/composables/useTimer.ts` - для логики обратного отсчета.

Такая разрозненность приводит к высокой когнитивной нагрузке и усложнению навигации по проекту.

Сложность удаления функционала

Прямым следствием низкого зацепления является высокая сложность проведения рефакторинга и безопасного удаления неактуального функционала. При изменении требований, в том случае, когда определенная фича более не требуется, ее удаление превращается в сложную задачу. Из-за отсутствия чётких границ между модулями разработчик не может быть уверен, что удаление глобального стора или некоторой утилиты не нарушит работоспособность других страниц. В результате в кодовой базе накапливается мертвый код, который продолжает собираться сборщиком, увеличивая размер итогового бандла приложения и замедляя его загрузку.

Сильная связность и круговые зависимости

Традиционная структура не накладывает никаких ограничений на направление импортов внутри проекта. Любой компонент может импортировать любой другой компонент, стор или утилиту. В условиях отсутствия вертикальной иерархии зависимостей компоненты начинают связываться напрямую. Это приводит к возникновению круговых зависимостей, когда модуль А импортирует модуль В, в свою очередь модуль В зависит от модуля А. Круговые зависимости вызывают трудноуловимые ошибки в рантайме, усложняют процесс сборки приложения и делают модули непереиспользуемыми. Изменение логики отображения элемента на одной странице может каскадно сломать логику на другой.

Отсутствие инкапсуляции и глубокие импорты

При традиционном подходе отсутствует понятие публичного контракта модуля. Компоненты импортируются напрямую. Любое перемещение файлов внутри папки `components` в

процессе рефакторинга ломает импорты по всему проекту. Отсутствие «фасадов» делает внутреннюю структуру компонентов открытой для внешнего использования, исключая возможность безопасной внутренней модификации кода.

Конфликты слияния в системе контроля версий

Поскольку все разработчики вынуждены обращаться к папкам общего назначения (добавлять новые файлы в единую папку components, регистрировать новые сторы в общем store), при совместной работе регулярно возникают конфликты слияния в Git. Разрешение таких конфликтов требует дополнительного времени и повышает риски случайного удаления чужого кода, что может поломать работоспособность системы. Кроме того, новый разработчик, приходя в проект, сталкивается с высокой сложностью освоения: ему требуется изучить весь проект целиком, чтобы понять взаимосвязи между разрозненными файлами.

Для разрабатываемой платформы обучения студентов, характеризующейся наличием непересекающихся ролевых интерфейсов (Администратор, Преподаватель, Студент), использование традиционного подхода к разработке клиентской части веб-приложения несет в себе некоторые риски [32]. Без жестких архитектурных правил код личного кабинета студента быстро окажется связанным с интерфейсом администратора через общие папки компонентов и стейта. Это сделает платформу хрупкой, усложнит процесс изолированного тестирования ролевых сценариев и полностью заблокирует возможность легкого последующего расширения функционала платформы.

2.2. Методология Feature-Sliced Design: основные понятия и структура слоев

Методология Feature-Sliced Design (FSD) [7,27] представляет собой современный архитектурный подход к проектированию и структурированию кодовой базы клиентских веб-приложений. В отличие от традиционных архитектурных паттернов, разделяющих файлы исключительно по техническому назначению, например, распределение по папкам: components, views, stores, FSD предлагает ориентировать структуру проекта на бизнес-логику и функциональные особенности продукта. Главная цель методологии заключается в снижении связности кода и повышении его сцепленности, что делает проект легко масштабируемым, поддерживаемым и устойчивым к регулярным изменениям требований.

Методология FSD базируется на следующих основных терминах: слои (папки верхнего уровня), слайсы (папки второго уровня) и сегменты (папки третьего уровня) (рис. 1)

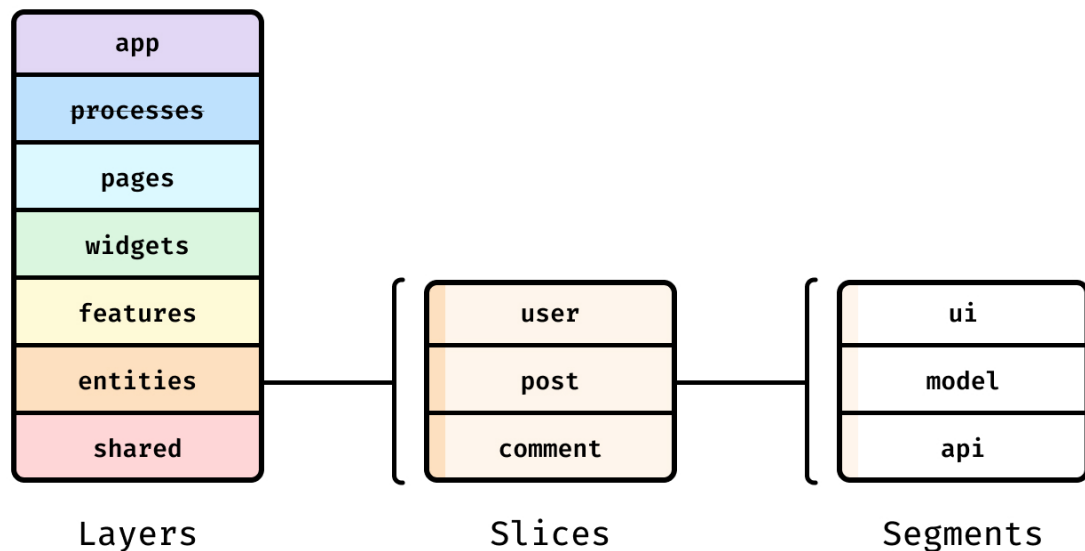


Рис. 1. Схема связи базовых концепций FSD

2.2.1. Слои

Слои представляют собой верхнеуровневые директории проекта и первый уровень организационной иерархии. В методологии FSD стандартизировано шесть основных слоев (ранее было 7, но слой process признан устаревшим), расположенных в строгой вертикальной иерархии от верхнего (системного) уровня к нижнему (инфраструктурному):

app - слой инициализации приложения. Здесь располагаются всё то, что касается приложения целиком. Этот слой обычно не содержит слайсов, а сразу использует сегменты. Пример того, что обычно располагается на этом слое:

- точка входа в приложение
- подключение глобальных стилей
- конфигурация маршрутизатора
- инициализация корневого стейт-менеджера
- общие провайдеры контекста

pages - слой страниц приложения. Слой содержит компоненты “экранов” (визуальных страниц), которые собираются из нижележащих блоков. Одна страница обычно соответствует

одному слайсу, однако, если есть несколько очень похожих страниц, их можно сгруппировать в один слайс, например, формы регистрации. Страницы на данном слое содержат минимум бизнес-логики и выполняют преимущественно композиционную роль. В слайсе страницы обычно располагается интерфейс страницы, а также состояния загрузки и границы ошибок, также допускается располагать здесь запросы на получение и изменение данных.

widjets - слой самостоятельных блоков интерфейса. Виджеты представляют собой крупные, функционально законченные модули, например, шапка, боковое меню, подвал сайта. Виджеты наиболее полезны, когда они используются на нескольких страницах или когда страница, к которой они принадлежат, имеет несколько больших независимых блоков, и это один из них.

features - слой пользовательских действий. Слой предназначен для основных взаимодействий в приложении, действий, которые важны пользователям. Эти взаимодействия часто затрагивают бизнес-сущности, поскольку сущности - это то, о чём приложение. Слайс фичи может содержать UI для выполнения действия, например, форму, вызовы API, необходимые для выполнения действия, валидацию и внутреннее состояние, фиче-флаги (некоторую конфигурацию).

entities - слой бизнес-сущностей предметной области. Слой содержит бизнес-сущности приложения. Обычно это термины, которые бизнес использует для описания продукта. Например, социальная сеть может работать с бизнес-сущностями, такими как Пользователь, Публикация и Группа. Слайс сущности может содержать хранилище данных, схемы валидации данных, функции запросов API, связанные с сущностями, а также визуальное представление этой сущности в интерфейсе.

shared - слой инфраструктурного кода. Слой формирует фундамент для остальной части приложения. Это место для создания связей с внешним миром, например, бэкенды, сторонние библиотеки, среда выполнения приложения. Также это место для собственных библиотек, сконцентрированных на конкретной задаче. Этим слоем, также как и App не содержит слайсов. Слайсы предназначены для разделения слоя на предметные области, но предметные области не существуют в Shared. Это означает, что все файлы в Shared могут ссылаться и импортировать друг друга. Пример того, что может располагаться на этом слое

- API-клиент и функции для выполнения запросов к конкретным эндпоинтам бэкенда
- UI-кит приложения
- коллекция внутренних библиотек

- переменные окружения, глобальные фиче-флаги и другая глобальная конфигурация для вашего приложения\
- код, настраивающий систему переводов, а также глобальные строки перевода

2.2.2. Слайсы

Слайсы - это второй уровень в организационной иерархии FSD. Их основное назначение заключается в группировке кода по его значению для продукта, бизнеса или просто приложения. Имена слайсов не стандартизированы, поскольку они напрямую определяются бизнес-областью приложения. Слои Shared и App не содержат слайсов. Это потому, что Shared не должен содержать никакой бизнес-логики, следовательно, не имеет продуктового значения, а App должен содержать только код, касающийся всего приложения, поэтому разделение не требуется

2.2.3. Сегменты

Сегменты - это третий и последний уровень в организационной иерархии, их цель - сгруппировать код по его техническому назначению.

Существует несколько стандартизированных названий сегментов:

- **ui** - все, что связано с отображением UI: UI-компоненты, форматтеры дат, стили и т.д.
- **api** - взаимодействие с бэкендом: функции запросов, типы данных, мапперы и т.д.
- **model** - модель данных: схемы, интерфейсы, хранилища и бизнес-логика.
- **lib** - библиотечный код, необходимый другим модулям в этом слайсе.
- **config** - конфигурационные файлы и фиче-флаги.

Также допустимо создавать свои собственные сегменты. Наиболее распространенные места для кастомных сегментов - это слои App и Shared, где слайсы не имеют смысла. При создании своего собственного сегмента необходимо убедиться, что название собственного сегмента описывает, для чего нужно его содержимое, а не чем оно является. Например, components, hooks и types - плохие названия сегментов, потому что они не так полезны при поиске кода.

2.3. Ключевые принципы FSD: публичный API, низкая связанность и высокое сцепление.

Архитектурная методология FSD основана на классических принципах проектирования программного обеспечения [32], сформулированных в рамках теории объектно-ориентированного программирования и системного анализа. Ключевыми метриками, определяющими качество архитектуры клиентского приложения, выступают слабая связанность и высокое сцепление, а также строгое соблюдение принципа инкапсуляции через механизмы публичного интерфейса. FSD предлагает инструменты и правила для достижения оптимальных значений этих метрик, что критически важно при разработке расширяемых и масштабируемых приложений.

2.3.1. Принцип публичного API

Публичный API - это контракт между группой модулей, например, слайсом, и кодом, который его использует. Он также действует как ворота, разрешая доступ только к определенным объектам и только через этот публичный API. На практике это обычно реализуется как индексный файл с реэкспортами.

Экспорт из слайса через `index.ts` решает несколько важных задач:

- **Инкапсуляция и сокрытие деталей реализации:** внешние модули не имеют доступа к внутренней структуре слайса. Например, если внутри слайса `features/submit-test` содержатся вспомогательные компоненты разметки, локальные функции валидации и специфические типы, они не экспортируются наружу. Это предотвращает случайное и некорректное использование внутренних утилит слайса в других частях приложения.
- **Безопасность рефакторинга:** разработчик может полностью изменить внутреннюю структуру слайса (переименовать файлы, разбить один крупный компонент на три мелких, переписать логику в сегменте `model`), не опасаясь нарушить работоспособность всего приложения. До тех пор, пока сигнатура экспортов в `index.ts` остается неизменной, внешние зависимости не потребуют корректировки.

- **Декларативность контрактов:** файл `index.ts` служит контрактом модуля, позволяя другому разработчику мгновенно понять, какие компоненты, сторы или методы данный слайс предоставляет для использования на других слоях.

2.3.2. Низкая связность

Связность (coupling) определяет степень взаимозависимости между различными программными модулями (рис. 2). В проектах без строгой архитектуры изменение логики в одном компоненте часто вызывает каскад непредсказуемых ошибок в других частях системы, которые не связаны с ним. FSD минимизирует связанность за счет двух фундаментальных ограничений:

- **Вертикальное разделение зависимостей.** Иерархическая структура слоев гарантирует, что изменения на верхних слоях, например, изменение дизайна страницы на слое `pages`, не затронут бизнес-логику на нижних слоях.
- **Запрет на перекрестные импорты.** Слайсы, расположенные на одном и том же слое, не могут импортировать код друг друга. Например, компонент отображения карточки студента не может напрямую импортировать и использовать компонент карточки курса. Если возникает необходимость объединить их функционал, это должно происходить на вышележащем слое.

Благодаря слабой связности модулей в системе достигается высокая изоляция ролевых интерфейсов. Изменения в личном кабинете студента гарантированно не затронут интерфейсы преподавателя или администратора, а обновление конкретной фичи, например, переход на новый плеер видеолекций, потребует изменений только внутри одного изолированного слайса.

2.3.3. Высокое зацепление

Зацепление (cohesion) характеризуется степенью нахождения кода, связанного между собой одной целью, в рамках одного модуля (рис. 2). Традиционная структура клиентских веб-приложений, разделяющая файлы по техническому типу (компоненты отдельно, хранилища отдельно, запросы к API отдельно), имеет крайне низкое зацепление. Для изменения одной фичи разработчик вынужден вносить правки в файлы, разбросанные по всему проекту.

FSD решает эту проблему путем доменно-ориентированного подхода. Все ресурсы, необходимые для функционирования конкретной бизнес-сущности или фичи (UI-компоненты,

стили, стейт-менеджеры, API-запросы, типы данных), группируются локально внутри одного слайса и распределяются по сегментам. В связи с этим можно выделить следующие преимущества высокого зацепления в рамках FSD:

- **Локализация контекста.** Разработчик, работающий над модулем, имеет всю необходимую логику, стили и запросы в рамках одной директории. Ему не требуется искать необходимые фрагменты кода по всему проекту.
- **Упрощение совместной разработки.** Поскольку работа над фичами локализована в разных слайсах, минимизируется вероятность возникновения конфликтов слияния в системе контроля версий Git при одновременной работе нескольких разработчиков над разными частями платформы.
- **Повышение масштабируемости.** Высокое зацепление гарантирует, что модуль представляет собой «черный ящик», который легко переносить, переиспользовать или удалять из проекта без риска оставить неиспользуемый («мертвый») код в глобальных папках стилей или стейта.

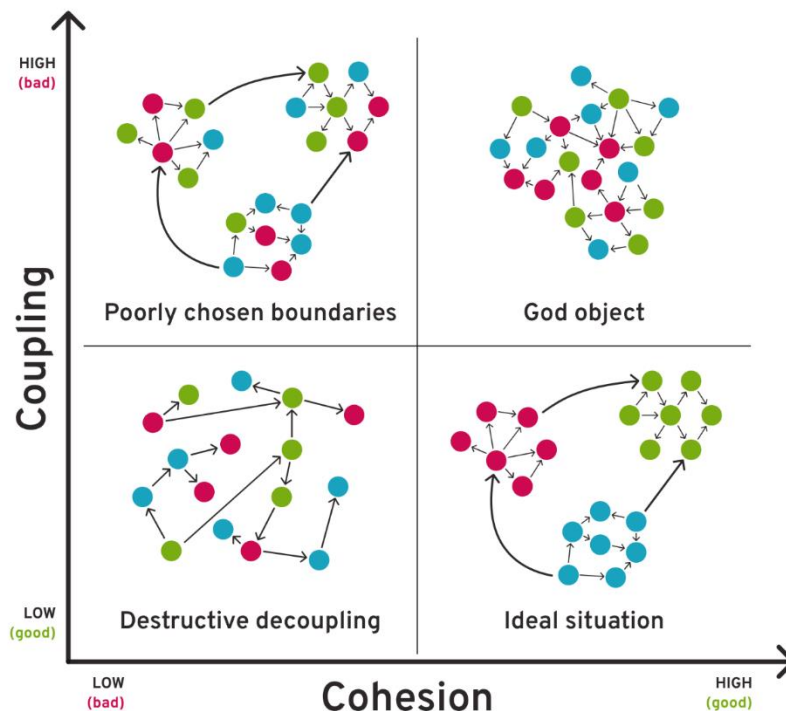


Рис. 2. Схема связи связности и зацепления в коде

2.4. Сравнительный анализ FSD с другими архитектурами фронтенд-приложений

Выбор архитектурного подхода для клиентского веб-приложения всегда представляет собой компромисс между строгостью структуры, скоростью разработки и сложностью поддержки кодовой базы. Для обоснования выбора FSD в рамках разработки платформы обучения студентов необходимо провести сравнительный анализ FSD с альтернативными архитектурными подходами, наиболее распространенными во фронтенд-разработке:

- Классическая (слоистая) архитектура по техническому типу (Layered/Flat Technical). Код группируется по ролям файлов (компоненты, сторы, роуты, утилиты).
- Доменно-модульная архитектура (Domain-Driven Modular): код группируется по бизнес-домена, например, папки /modules/course, /modules/user, но отсутствует строгое вертикальное деление на слои, а связи между модулями не регламентированы.
- Чистая архитектура (Clean Architecture). Перенесенный с бэкенд-разработки паттерн Роберта Мартина [32], делящий код на концентрические круги (доменные сущности, юзкейсы, адаптеры интерфейсов, внешние фреймворки и UI).

Для удобства сравнения и выявления наиболее сбалансированного решения данные подходы сопоставлены по ключевым критериям качества программного обеспечения (табл. 2).

Таблица 2.

Сравнительный анализ архитектурных подходов для фронтенд-приложений

Критерий сравнения	Layered/Flat Technical	Domain Modular	Clean Architecture	Feature-Sliced Design
Слабая связанность	Очень низкая (хаотичные связи)	Средняя (риск cross-imports)	Высокая	Высокая (регулируется слоями)
Зацепление кода	Низкое	Высокое	Высокое	Высокое (через слайсы и сегменты)

Масштабируемость кода	Низкая	Средняя	Высокая	Высокая
Порог входа	Минимальный	Средний	Очень высокий	Средний
Объем шаблонного кода	Минимальный	Средний	Избыточно высокий	Умеренный
Применимость для UI-фреймворков	Высокая	Высокая	Низкая (требует адаптеров)	Высокая (UI-first подход)
Простота расширения	Низкая	Средняя	Высокая	Высокая

Сравнение FSD с альтернативными подходами позволяет выявить ключевые преимущества и недостатки каждого подхода в контексте решаемой задачи.

Традиционная структура привлекает разработчиков отсутствием шаблонного кода и высокой скоростью создания базового прототипа. Однако по мере добавления новых требований в продукте приложение в плоской структуре быстро превращается в монолитный узел неявных зависимостей. FSD же требует чуть больше времени на начальную настройку слоев и сегментов, но взамен гарантирует стабильную скорость разработки и безопасность рефакторинга на протяжении всего жизненного цикла продукта.

Доменно-модульный подход решает проблему низкого зацепления, группируя все файлы, связанные одним логическим назначением в одну директорию. Главный недостаток этого подхода - отсутствие вертикальной иерархии. Без правил, запрещающих перекрестные импорты, модули неизбежно начинают зависеть друг от друга напрямую. Это приводит к циклическим зависимостям. FSD устраняет эту проблему за счет введения вертикальных слоев, строго определяющих направление потока зависимостей только сверху вниз.

Чистая архитектура предлагает максимальную независимость бизнес-логики от внешних фреймворков и UI. Однако при переносе этого паттерна на фронтенд возникает избыточное

усложнение. Современные клиентские приложения, разработанные на фреймворках реактивны по своей природе и тесно связаны с DOM. Попытка изолировать доменные сущности от реактивного состояния, создавать адаптеры данных и выделять отдельные слои репозитория для простых CRUD-запросов приводит к лавинообразному росту шаблонного кода. В отличие от Clean Architecture, методология FSD ориентирована непосредственно на клиентскую веб-разработку. Она предлагает декомпозицию, где реактивный стейт-менеджер и API-клиент органично интегрируются в сегменты `model` и `api` каждого слайса, не требуя создания лишних слоев абстракции.

Таким образом, сравнительный анализ [7] показывает, что FSD представляет собой оптимальный баланс между модульной строгостью, скоростью разработки и простотой поддержки. Для платформы обучения студентов выбор FSD обеспечивает масштабируемость и дает гарантию того, что новые функциональные модули, создаваемые в будущем, могут быть интегрированы в систему в качестве изолированных слайсов без риска разрушить существующие связи в приложении.

3. Инструментальные средства разработки

Перед рассмотрением прикладных задач, которые были решены в рамках работы, необходимо сначала описать выбранные инструментальные средства разработки. Это поможет читателю понять, какие технологии были использованы для решения задач и какие возможности были доступны для автора работы. Также это поможет другим специалистам в данной области оценить применимость выбранных инструментов и возможность их использования в своих проектах.

Для реализации интерфейса, управления данными, маршрутизации и сопровождения жизненного цикла разработки был сформирован программный стек на базе экосистемы Vue 3. Материал главы организован по функциональным блокам: платформа сборки, ядро приложения, управление состоянием и навигация, построение интерфейса, работа с данными и формами, инструменты качества кода, а также окружение разработки и развертывания.

Описание каждой технологии строится по единому шаблону:

1. **Назначение** - краткая характеристика инструмента и его роли в современной frontend-разработке.
2. **Решаемые проблемы** - какие задачи проекта обусловили необходимость его применения.
3. **Обоснование выбора** - сравнение с рассмотренными альтернативами и аргументы в пользу принятого решения.
4. **Применение в проекте** - конкретные места использования в кодовой базе (модули, сценарии, команды).
5. **Иллюстрации** (при необходимости) - схемы, таблицы или скриншоты, поясняющие интеграцию инструмента.

Сводная характеристика всего стека приведена в таблице 3 в конце главы.

3.1. Сборка

3.1.1. Vite

Vite [19] - инструмент сборки и dev-сервера для современных фронтенд-приложений. В режиме разработки он использует нативные ES-модули браузера и подает исходный код по

необходимости, не собирая всё приложение целиком при каждом старте. Для production выполняется оптимизированная сборка на базе инструмента Rollup (рис. 3).

Решаемые проблемы. Без специализированного сборщика пришлось бы вручную настраивать транспилицию TypeScript, обработку .vue-файлов, hot module replacement (HMR), алиасы путей и проксирование API. Классические связки Webpack + loader'ы дают более медленный цикл разработки на проектах.

Обоснование выбора. Vite официально рекомендуется для проектов Vue 3, хорошо интегрируется с TypeScript, Vitest и Storybook. Альтернатива:

- Webpack (Create Vue App) - уступает по скорости dev-сервера
- Nuxt избыточен для SPA без SSR.

Применение в проекте. Конфигурация задаётся в файле vite.config.ts. Плагин @vitejs/plugin-vue для SFC, алиас @/ → src/, проху /api на бэкенд с удалением префикса - поведение, повторённое позже в nginx для production. Сборка выполняется командой npm run build, результат помещается в каталог dist/.

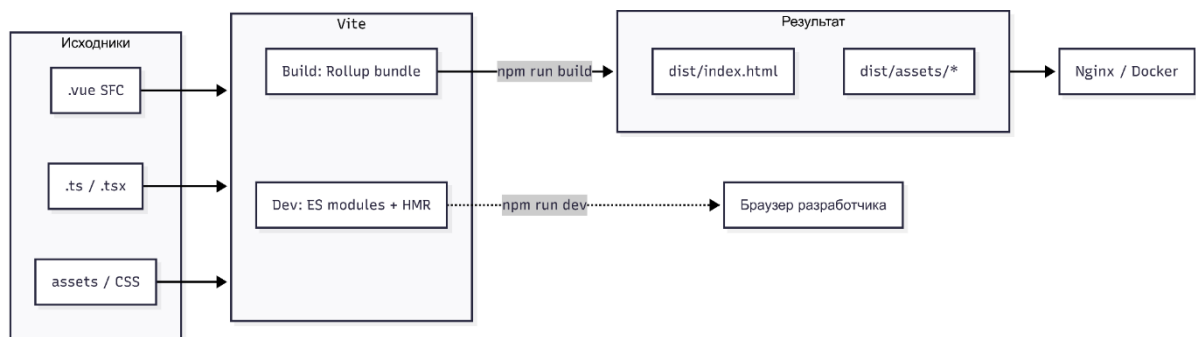


Рис. 3. Схема работы сборщика Vite

3.2. Ядро приложения

3.2.1. Vue 3 (composition API)

Vue 3 [11] - прогрессивный фреймворк для построения пользовательских интерфейсов. Composition API - способ организации логики компонента через функции (setup, composables) вместо опций объекта (Options API).

Решаемые проблемы. Composition API упрощает выделение переиспользуемой логики (авторизация, работа с формами, запросы), улучшает типизацию в TypeScript и делает код

крупных компонентов читаемым: связанная логика группируется рядом, а не разносится по блокам data, methods, computed.

Обоснование выбора. Vue 3 выбран как основной UI-фреймворк проекта; Composition API используется во всех компонентах (`<script setup lang="ts">`). Сравнительный анализ Vue, React и Angular представлен в работах [5,6]. React потребовал бы иной экосистемы и архитектурных решений; Angular - более высокого порога входа для приложения учебной платформы

Применение в проекте. Интерфейсы трёх ролей (администратор, преподаватель, студент) реализованы как набор Vue-компонентов с декларативными шаблонами. Логика вынесена в composables и Pinia-stores. Макросы Vue 3.4+ (defineProps, defineEmits, defineModel) применяются для типобезопасного API компонентов.

3.2.2. TypeScript

TypeScript [20] - типизированное надмножество JavaScript (рис. 4), компилируемое в обычный JS. Ошибки типов выявляются на этапе разработки и сборки, а не только в runtime.

Решаемые проблемы. В проекте с десятками сущностей (пользователь, курс, assessment, организационная иерархия) без статической типизации высок риск несогласованности данных между API, store и UI. TypeScript фиксирует контракты: DTO ответов бэкенда, доменные типы, props компонентов.

Обоснование выбора. TypeScript - стандарт фронтенд приложений; поддерживается vue-tsc, Vite, ESLint. В конфигурации включён режим noUncheckedIndexedAccess, что требует явной проверки индексов массивов и ключей объектов - повышает надёжность при работе с динамическими данными.

Применение в проекте. Типы сущностей описаны в слое entities (User, Course, StudentCourse и др.). API-ответы мапятся в доменные модели (mapAdminUserListItemToUser). Абсолютные импорты через @/ согласуются с path mapping в tsconfig.app.json.

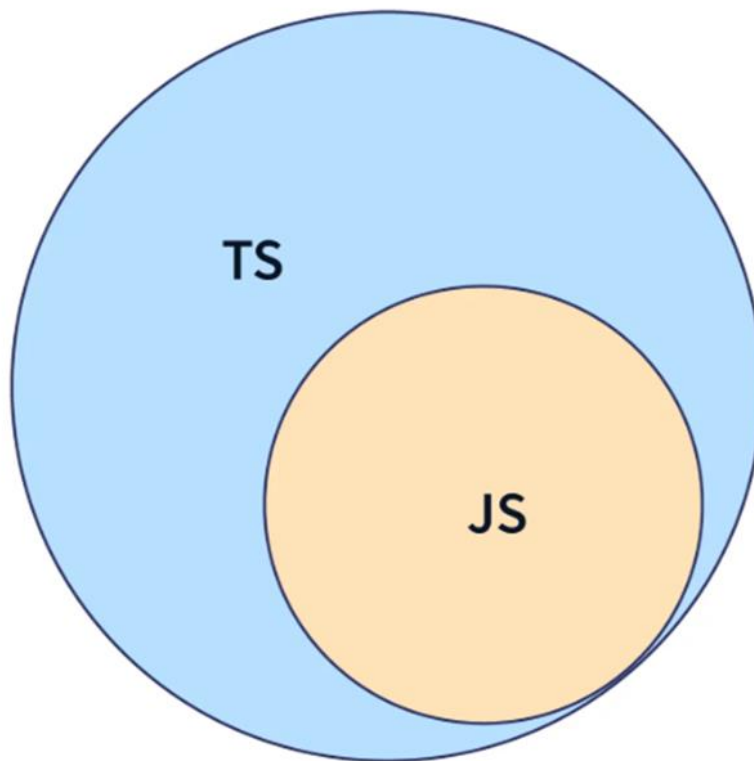


Рис. 4. Соотношение языков JS и TS

3.3. Состояние и навигация

3.3.1. Pinia

Pinia [13] - официальное решение для управления состоянием в Vue 3, преемник Vuex с более простым API и полной поддержкой TypeScript.

Решаемые проблемы. Часть состояния должна быть доступна глобально: сессия пользователя, тема оформления, локальные UI-флаги. Передача таких данных через props по всему дереву компонентов не масштабируется.

Обоснование выбора. Pinia рекомендован документацией Vue 3; stores описываются как composable-функции (defineStore + setup-синтаксис), что согласуется с Composition API. Vuex устаревает; Context/Provide подходит только для узких сценариев.

Применение в проекте. useSessionStore хранит факт авторизации и роль пользователя (из JWT). useThemeStore управляет светлой/тёмной темой. Дополнительные stores в слое entities инкапсулируют состояние отдельных доменных областей. Серверные данные (списки курсов,

пользователей) сознательно не дублируются в Pinia - для них используется TanStack Query (рис. 5).

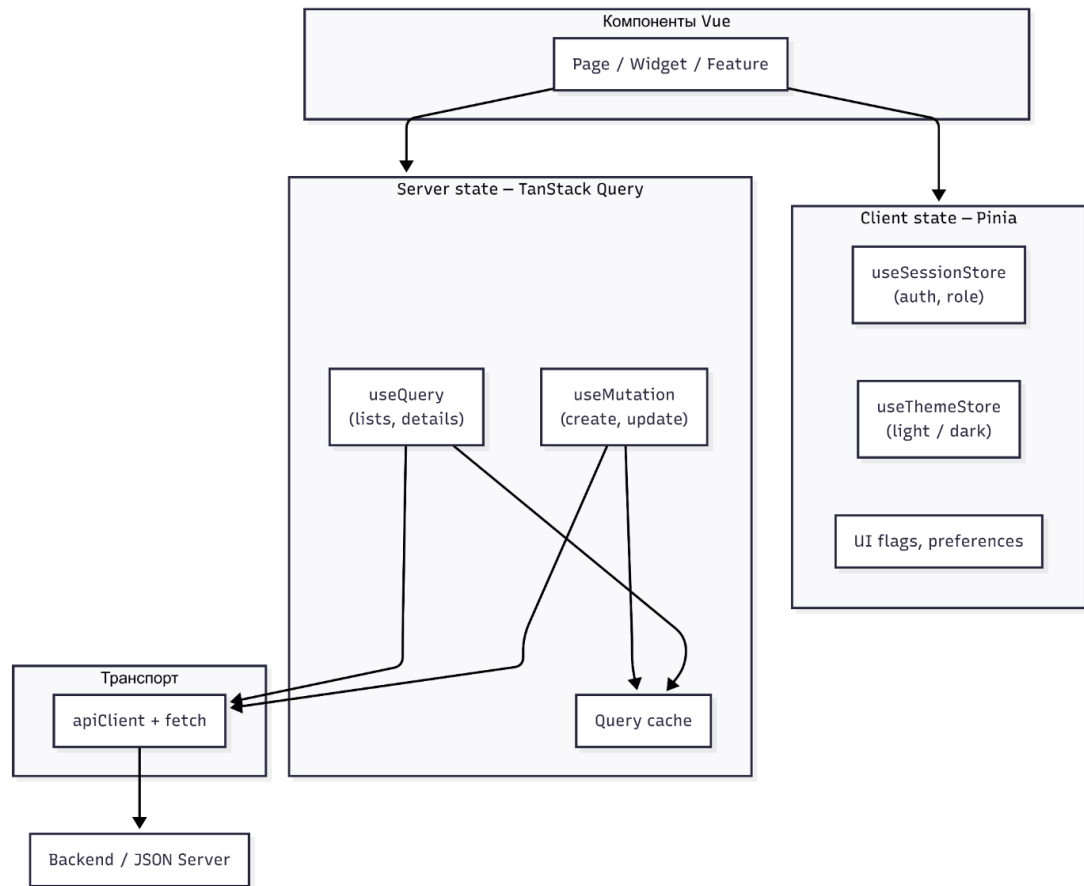


Рис. 5. Схема разделения состояния приложения на серверное и клиентское

3.3.2. Vue router

Vue Router [12] - официальная библиотека маршрутизации для Vue, обеспечивающая навигацию в SPA без перезагрузки страницы.

Решаемые проблемы. Платформа имеет три изолированных «портала» (/admin, /teacher, /student) с вложенными маршрутами (курсы, профили, прохождение тестов). Нужны lazy loading страниц, защита маршрутов по авторизации и роли (RBAC) [8], редирект после логина.

Обоснование выбора. Vue Router интегрирован с Vue 3, поддерживает nested routes, navigation guards, programmatic navigation. Альтернативы (сторонние router'ы) не дают преимуществ в экосистеме Vue.

Применение в проекте. Маршруты объявлены в app/providers/router/index.ts. Глобальный beforeEach guard (guards.ts) проверяет requiresAuth, requiresGuest и allowedRoles на основе

useSessionStore. Студентский flow включает цепочку: курс → информация об assessment → сессия прохождения (student-assessment-take).

3.4. Интерфейс

3.4.1. PrimeVue

PrimeVue [15] - библиотека UI-компонентов для Vue с набором готовых элементов: таблицы, формы, модальные окна, уведомления, деревья и др.

Решаемые проблемы. Разработка админ-панели, таблиц пользователей, форм создания курса и assessment UI «с нуля» потребовала бы значительных трудозатрат и не гарантировала единообразия и базовой доступности (a11y).

Обоснование выбора. PrimeVue 4 поддерживает Vue 3, темизацию приложения через design tokens, тёмную тему через darkModeSelector. Альтернативы: Vuetify (Material Design, другая визуальная система), Element Plus (менее гибкий theming). PrimeVue выбран за богатый набор «enterprise»-компонентов, подходящих для учебной платформы.

Применение в проекте. Глобально подключены PrimeVue, ToastService, ConfirmationService в main.ts. Компоненты Button, InputText, DataTable, Select, Tag, Dialog используются в features и widgets. Тема Aura согласована с собственными design tokens (tokens.css); переключение light/dark синхронизировано с useThemeStore.

3.4.2. Vue i18n

Vue I18n [14]- библиотека интернационализации для Vue: перевод строк, переключение локали.

Решаемые проблемы. Интерфейс платформы должен поддерживать как минимум русский и английский языки. Хардкод строк в шаблонах затрудняет поддержку и расширение локалей.

Обоснование выбора. Vue I18n - стандарт de facto для Vue; интегрируется с Composition API (useI18n). Альтернатива - ручные словари без библиотеки - не масштабируется.

Применение в проекте. Конфигурация в `shared/config/i18n`: файлы `en.json`, `ru.json`. Компонент `LanguageSwitcher` переключает локаль и сохраняет выбор в `localStorage`. Дополнительно синхронизируется локаль `PrimeVue` (календари, фильтры) через `getLocaleMessage` (рис. 6).

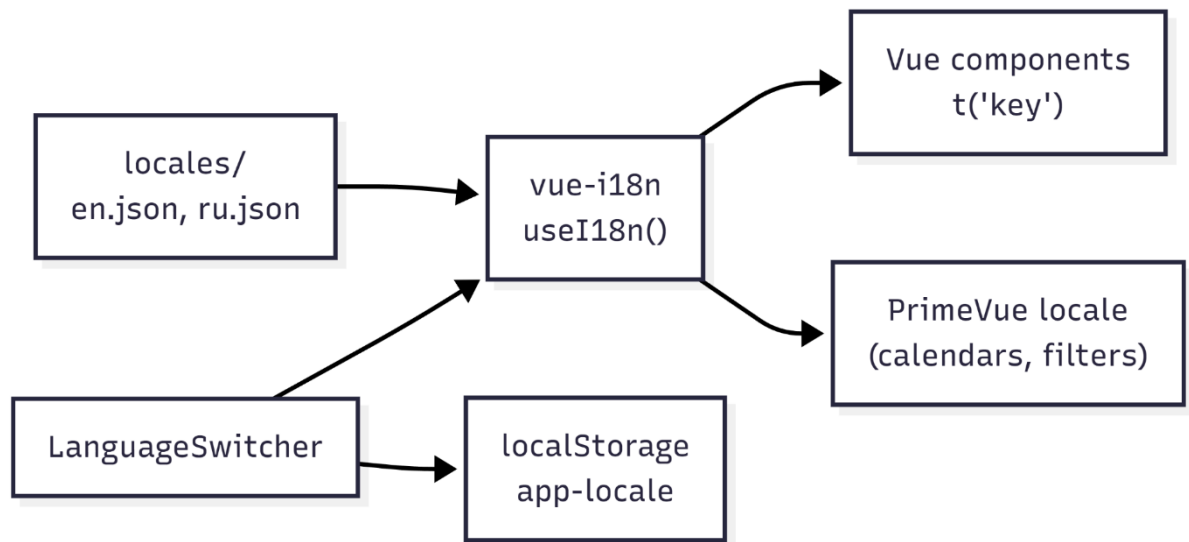


Рис. 6. Схема организации локализации приложения

3.5. Данные и формы

3.5.1. TanStack Query (Vue query)

TanStack Query (Vue Query) [16] - библиотека для управления серверным состоянием: запросы к API, кэширование, фоновое обновление, обработка `loading/error`.

Решаемые проблемы. Без централизованного слоя каждый компонент самостоятельно вызывал бы `fetch`, дублировал бы логику загрузки и не переиспользовал бы уже полученные данные. При навигации между страницами курсов это приводило бы к лишним запросам и «миганию» UI.

Обоснование выбора. TanStack Query - стандарт для работы с серверными данными в клиентском приложении; Pinia при этом остаётся для `client state`. SWR/React Query аналогичны, но TanStack имеет официальный Vue-адаптер. «Голый» `fetch` + Pinia для всех данных смешивал бы два типа состояния в одном store.

Применение в проекте. VueQueryPlugin подключён в main.ts. API-функции в слое entities/*/api вызываются из queryFn / mutationFn. Devtools включены для отладки. Списки курсов, пользователей, банков вопросов - типичные сценарии использования.

3.5.2. Vee-validate и zod

Vee-Validate [17] - библиотека валидации форм для Vue. **Zod** [18]- декларативная схема валидации и парсинга данных на TypeScript с выводом типов.

Решаемые проблемы. Формы платформы (авторизация, добавление пользователя, создание курса) требуют проверки полей на клиенте до отправки на сервер. Разрозненные if в обработчиках submit сложно поддерживать; правила валидации должны совпадать с типами payload для API.

Обоснование выбора. Связка @vee-validate/zod позволяет описать схему один раз в Zod и использовать её и для валидации UI, и для типизации (z.infer). Альтернатива - только Vee-Validate с rule-объектами без единой схемы - хуже для сложных форм с условными правилами (например, обязательность departmentId только для роли Teacher).

Применение в проекте. Пример: feature add-user- createAddUserFormSchema(t) в schema.ts с superRefine для role-specific правил; buildCreateAdminUserPayload формирует тело запроса после успешной валидации. Unit-тесты покрывают и схему, и сборку payload.

3.6. Качество и разработка

3.6.1. Vitest

Vitest [21]- test runner, ориентированный на проекты Vite: общая конфигурация, быстрый запуск, API, совместимый с Jest.

Решаемые проблемы. Необходимо автоматически проверять корректность бизнес-логики (мапперы, схемы, сервисы) при изменениях кода. Ручное тестирование регрессий в проекте с десятками экранов трудоемко.

Обоснование выбора. Vitest использует тот же pipeline, что Vite; не требует отдельной тяжёлой конфигурации Jest для ESM и Vue. Окружение happy-dom даёт DOM API для тестов, зависящих от окружения браузера, например, LocalStorage.

Применение в проекте. Конфигурация в `vitest.config.ts` с проектом `unit` (unit-тесты) и проектом `storybook` (тесты `stories` через `Playwright`)

3.6.2. Storybook

Storybook [23] - среда для изолированной разработки и документирования UI-компонентов вне основного приложения.

Решаемые проблемы. Компонент внутри страницы сложно разрабатывать и демонстрировать: нужна авторизация, навигация, загрузка данных. Storybook позволяет рендерить компонент с `mock-props` и глобальными провайдерами (`Pinia`, `i18n`, `PrimeVue`).

Обоснование выбора. Storybook - стандарт для `design systems` и `component-driven development`. Интеграция с `Vue 3 + Vite` официально поддерживается. Аддоны `@storybook/addon-all` и `@storybook/addon-vitest` закладывают основу для `accessibility`- и `visual`-тестирования без отдельного E2E на раннем этапе.

Применение в проекте. Stories расположены рядом с компонентами (`WidgetCard.stories.ts`, `CourseCard.stories.ts` и др.). В `.storybook/preview.ts` подключены `design tokens` (`tokens.css`), `PrimeVue`, `i18n` - визуальное `stories` соответствует приложению.

3.6.3. ESLint и Prettier

ESLint [24] - линтер для `JavaScript/TypeScript`; **eslint-plugin-vue** - правила для `Vue SFC`. **Prettier** [25] - форматтер кода/файлов.

Решаемые проблемы. В командной разработке важны единообразие стиля и раннее обнаружение ошибок (неиспользуемые переменные, неверное использование `Vue API`). Без автоматизации качество кода зависит только от дисциплины разработчика.

Обоснование выбора. Связка `ESLint + Prettier` - стандарт `Vue/TypeScript` проектов. `eslint-config-prettier` исключает конфликты правил форматирования.

Применение в проекте. Скрипт `npm run lint` запускает `eslint` с `--fix`. `npm run format` форматирует `src/` через `Prettier`. Конфигурация `ESLint` - `eslint.config.ts` (flat config) (рис. 7).

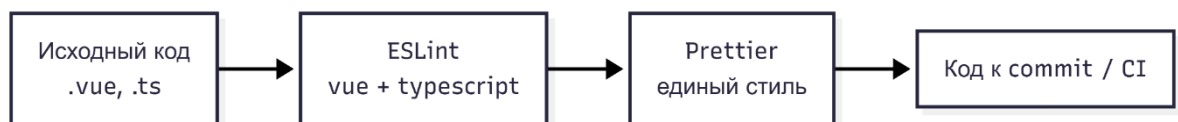


Рис. 7. Схема организации стилистики кода

3.6.4. Husky

Husky [26]- инструмент для настройки Git hooks: скрипты, автоматически выполняемые при commit, push и др.

Решаемые проблемы. Без hooks разработчик может закоммитить код с ошибками линтера, непрогнанными тестами или неверным форматированием. Принцип раннего обнаружения дефектов - один из ключевых в обеспечении качества программного обеспечения [32].

Обоснование выбора. Husky - наиболее распространённое решение для проектов; проще альтернативных решения в виде ручных pre-commit скриптов или Lefthook. Планируется связка с lint-staged для проверки только изменённых файлов - сокращение времени hook'a.

Применение в проекте. Hook pre-commit: lint-staged → Prettier + ESLint на staged-файлах; опционально npm run test:run на pre-push. Это замыкает цепочку качества: локально (Husky) → CI → production.

3.7. Окружение и развертывание

3.7.1. Json-server

JSON Server - утилита для быстрого развёртывания REST API поверх файла JSON. Она интерпретирует коллекции в db.json как ресурсы (`/users`, `/courses` и т.д.) и автоматически поддерживает операции GET, POST, PUT, PATCH, DELETE, а также фильтрацию и пагинацию через query-параметры.

Решаемые проблемы. На ранних этапах создания клиентской части платформы полноценный бэкенд может быть недоступен, нестабилен или реализован частично. При этом frontend уже должен:

- отображать списки пользователей, курсов, организационную иерархию;
- обрабатывать формы создания и редактирования (POST/PATCH);
- отрабатывать сценарии авторизации и обработки ошибок сети.

Без mock API разработчик вынужден либо определять данные в компонентах, либо писать объемный перехватчик запросов внутри клиентского кода. Оба подхода усложняют поддержку и не соответствуют реальному HTTP-взаимодействию. JSON Server выносит mock-данные в

отдельный процесс, что позволяет клиентской части работать с тем же fetch или TanStack Query, что и в production, меняя только адрес API через механизм проху в сборщике Vite (рис. 8).

Обоснование выбора.

Таблица 3.

Сравнительный анализ инструментов для предоставления API

<i>Критерий</i>	<i>JSON Server</i>	<i>MSW (Mock Service Worker)</i>	<i>Mock в коде клиента</i>
<i>Где выполняется mock</i>	<i>Отдельный Node.js-процесс</i>	<i>Service Worker в браузере</i>	<i>Внутри apiClient</i>
<i>REST из коробки</i>	<i>Да</i>	<i>Нужно описывать handlers</i>	<i>Нужно писать вручную</i>
<i>Редактирование данных</i>	<i>db.json + сохранение при POST</i>	<i>In-memory в handlers</i>	<i>In-memory в коде</i>
<i>Близость к реальному API</i>	<i>Высокая</i>	<i>Высокая</i>	<i>Средняя</i>
<i>Сложность настройки</i>	<i>Низкая</i>	<i>Средняя</i>	<i>Растёт с числом эндпоинтов</i>

MSW удобен для тестов и изоляции компонентов, но для разработки экранов с десятками эндпоинтов JSON Server проще сопровождать: достаточно обновить JSON-файл.

Mock в коде клиента (перехват в `apiClient`) смешивает инфраструктуру и бизнес-слой, увеличивает размер бандла и усложняет переключение на реальный бэкэнд.

Вывод. JSON Server выбран как основной инструмент локального mock API на этапе разработки frontend.

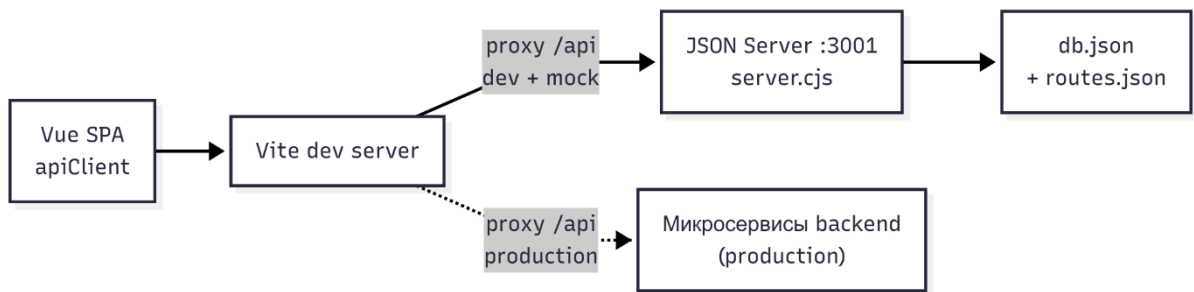


Рис. 8. Схема организации работы с несколькими API

3.7.2. Docker и nginx

Docker [28] и **Nginx** [29] образуют производственный стек развёртывания приложения. **Решаемые проблемы.** Production-сборка должна воспроизводимо собираться в одинаковом окружении (рис. 9) и раздаваться с поддержкой SPA-маршрутизации (fallback на index.html) и проксирования API (рис. 10).

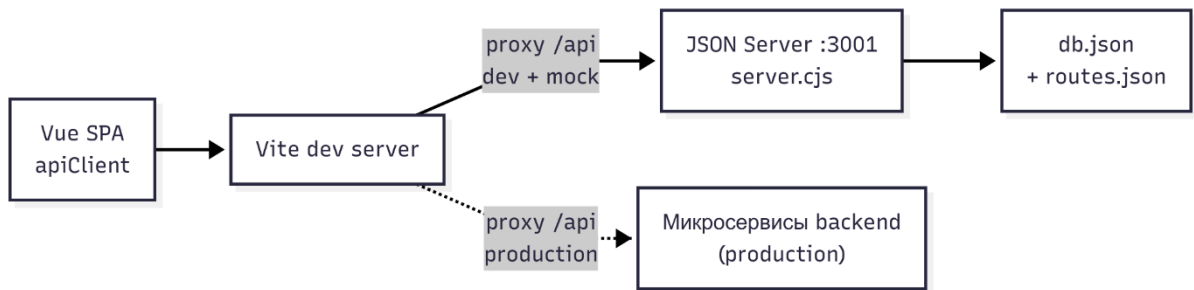


Рис. 9. Схема работы приложения при развёртывании в docker

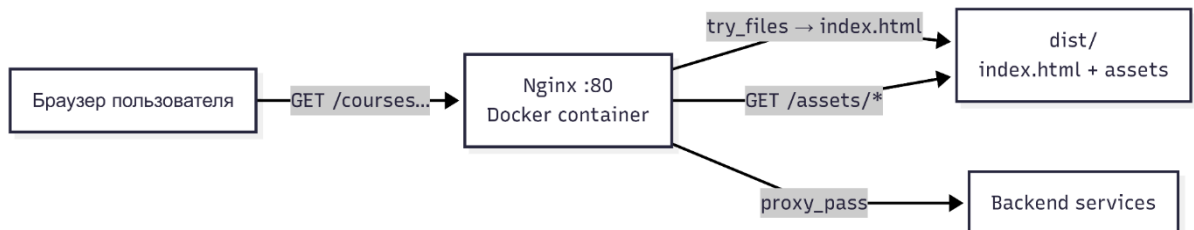


Рис. 10. Схема работы веб-сервера Nginx

3.8. Описание результатов

В данном пункте представлена сводная таблица используемого программного стека.

Таблица 4.

Сводная таблица используемых технологий

<i>№</i>	<i>Технология</i>	<i>Категория</i>	<i>Назначение в проекте</i>	<i>Рассмотренные альтернативы</i>	<i>Причина выбора</i>
<i>1</i>	<i>Vite</i>	<i>Сборка</i>	<i>Dev-сервер, HMR, production build</i>	<i>Webpack, Parcel</i>	<i>Скорость разработки, нативная поддержка Vue/TS</i>
<i>2</i>	<i>Vue 3</i>	<i>UI-фреймворк</i>	<i>Компоненты, Composition API</i>	<i>React, Angular</i>	<i>Опыт использования, согласованная экосистема</i>
<i>3</i>	<i>TypeScript</i>	<i>Типизация</i>	<i>DTO, domain types, props</i>	<i>JavaScript</i>	<i>Раннее обнаружение ошибок, упрощение рефакторинга</i>
<i>4</i>	<i>Pinia</i>	<i>Client state</i>	<i>Сессия, тема, UI-состояние</i>	<i>Vuex, composables-only</i>	<i>Официальный стандарт Vue 3, TS поддерживается из коробки</i>
<i>5</i>	<i>Vue Router</i>	<i>Маршрутизация</i>	<i>RBAC, три портала ролей</i>	<i>-</i>	<i>Стандарт для Vue</i>
<i>6</i>	<i>PrimeVue</i>	<i>UI-kit</i>	<i>Таблицы, формы,</i>	<i>Vuetify, Element Plus</i>	<i>Богатый набор компонентов,</i>

			модалки, Toast		доступна кастомизаци я при помощи css токенов
7	Vue I18n	Локали зация	RU/EN интерфейс	ручные словари	Станд арт для Vue
8	TanStack Vue Query	Server state	Кэш API, mutations, loading/error	SWR, ручной fetch + Pinia	Разделение client/server state. Уменьшение написания однотипной логики
9	Vee-Validate + Zod	Валидация форм	Unit-тесты логики	Yup, только Vee-Validate	Единая Zod- схема + вывод типов
10	Vitest	Тестирование	Unit-тесты логики и компонентов	Jest	Общий pipeline с Vite
11	Storybook	UI- документаци я	Stories, a11y, visual-тесты	Ladle, Histoire	Экосис тема, addon- vitest
12	ESLint + Prettier	Качество кода	линтинг и форматтер	Biome	Стандарт индустрии
13	Husky	Git hooks	Pre- commit/pre- push проверки	самописные проверки	Стандарт проектов
14	JSON Server	Dev- окружение	REST mock API из JSON	MSW, mock- client в коде	Отдельный сервер, CRUD из коробки

<i>15</i>	<i>Docker + Nginx</i>	<i>Deploy</i>	<i>Сборка образа, SPA, proxy /api</i>	<i>-</i>	<i>Стандарт индустрии</i>
-----------	---------------------------	---------------	---	----------	-------------------------------

4. Разработка клиентского программного модуля платформы на основе методологии FSD

В данном разделе рассматривается процесс разработки клиентской части платформы, который обеспечивает для пользователей графический интерфейс. Результатом этой работы будет готовое клиентское веб-приложение, которое можно будет использовать в составе разрабатываемой платформы.

4.1. Архитектурная структура проекта и декомпозиция по слоям FSD

Реализация клиентской части платформы обучения студентов выполнена в соответствии с архитектурной методологией FSD, теоретические основы которой были детально проанализированы в разделах 2.2 и 2.3. Применение данного подхода позволило распределить кодовую базу разрабатываемого Vue-приложения по независимым слоям, слайсам и сегментам, полностью решив проблему сильной связности модулей, свойственную традиционным подходам к фронтенд-разработке.

Для наглядного представления иерархической организации разработанной платформы ниже приведена часть структуры директорий внутри директория `src` исходного кода проекта (рис. 11):

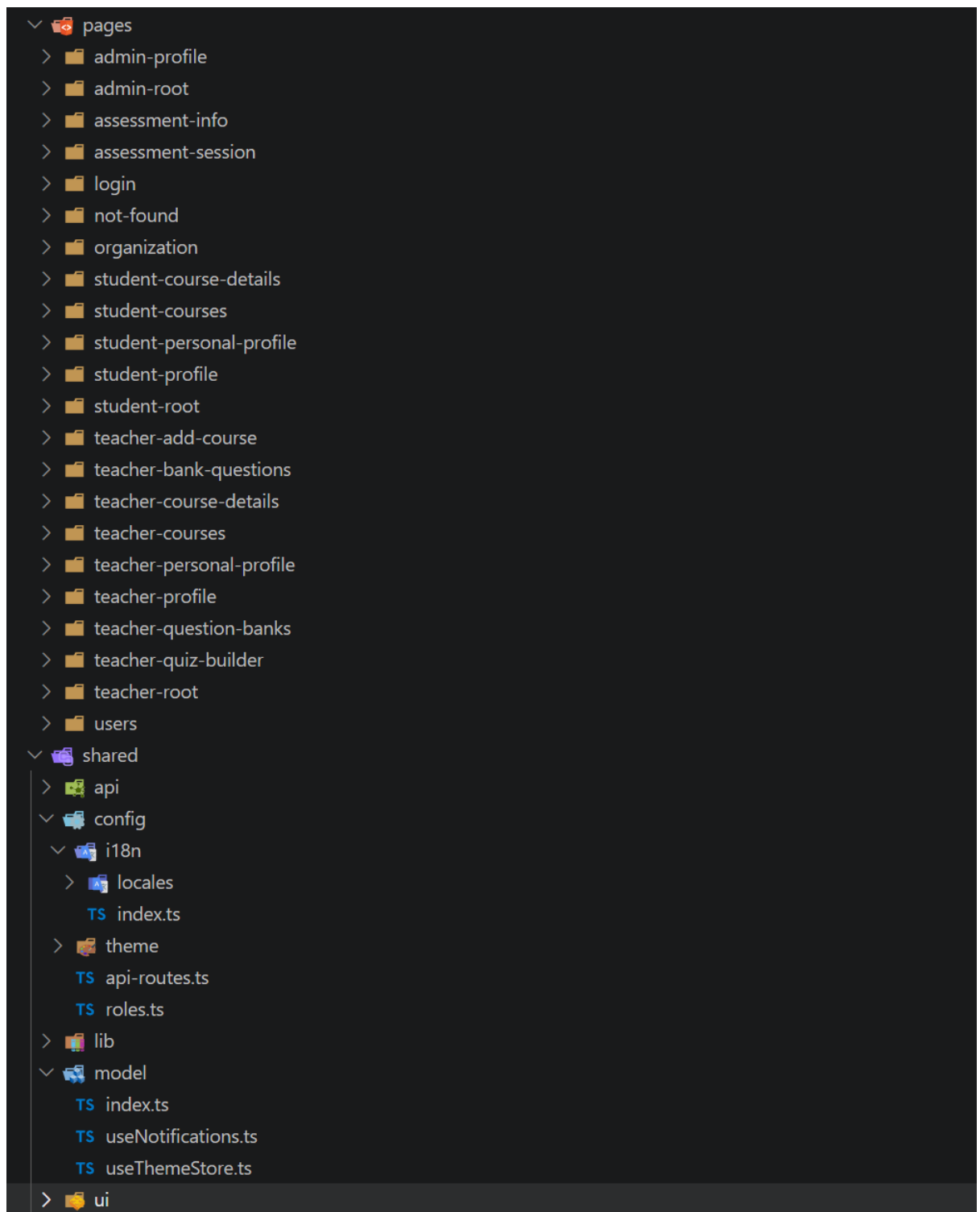


Рис. 11. Слои pages и shared приложения

Декомпозиция кодовой базы платформы по слоям FSD в процессе реализации выполнялась следующим образом:

4.1.1. Слой shared

На данном слое сосредоточен переиспользуемый код, полностью лишенный привязки к конкретной бизнес-логике. В сегменте `shared/ui` реализованы некоторые базовые компоненты интерфейса и обёртки над компонентами библиотеки компонентов PrimeVue (лейауты, карточки, кнопки). В сегменте `shared/api` сконфигурирован базовый клиент `base-client.ts` на основе функции `fetch`, которая предоставляется окружением браузера, содержащий общую настройку перехватчиков (`interceptors`) для обработки сетевых ошибок и автоматического обновления JWT-токенов [30]. В сегменте `shared/config` располагаются файлы с константами, которые используются на различных уровнях приложения, например, роли, которые доступны в приложении и переводы для языков, поддерживаемых в приложении.

4.1.2. Слой entities

Этот слой содержит описание структуры данных и базовые компоненты для работы с ключевыми объектами предметной области. Ниже приведены описания для некоторых из используемых сущностей приложения:

entities/user - содержит TypeScript-интерфейсы пользователя (роли: администратор, преподаватель, студент; статусы), методы получения и изменения данных через Users API (список, создание, редактирование, удаление, смена пароля и аватара) и Pinia-стор с кэшем пользователей по идентификатору.

entities/course - содержит TypeScript-интерфейсы учебного курса (курс преподавателя, курс студента, элементы содержания), методы получения и изменения данных через Course API, UI-компоненты карточек курса (`CourseCard`, `StudentCourseCard`) для сетки дашборда и два стора: курсы преподавателя и курсы студента.

entities/question-bank - содержит TypeScript-интерфейсы банка вопросов и отдельного вопроса (типы, варианты ответов, форма редактирования), методы CRUD для банков и вопросов и стор с кэшем загруженных банков.

entities/organization - содержит TypeScript-интерфейсы административной и академической структуры университета (факультет, кафедра, направление, группа), методы получения дерева и CRUD подразделений, мапперы DTO и стор с ленивой подгрузкой дочерних узлов.

entities/assessment-session - содержит TypeScript-интерфейсы сессии прохождения контрольной работы (вопросы, ответы, таймер), методы загрузки сессии и автосохранения ответов и стор, управляющий текущим вопросом, прогрессом, таймером и флагами «на проверку».

4.1.3. Слой features

На слое фич сосредоточены изолированные интерактивные действия, несущие непосредственную бизнес-ценность и изменяющие состояние системы. Слайсы этого слоя собираются из сущностей и общих компонентов. Ниже приведены описания для некоторых из используемых фич приложения:

features/auth-by-email - инкапсулирует форму авторизации по email и паролю, включая валидацию полей (Vee-Validate и Zod), опцию «запомнить меня» и асинхронную отправку запроса на аутентификацию с перенаправлением в портал по роли.

features/add-org-unit - инкапсулирует модальное окно создания подразделения оргструктуры (тип, название, родительский узел) с валидацией формы и вызовом API создания через entity organization.

features/add-user - инкапсулирует сценарий создания пользователя администратором: кнопку вызова и модальное окно с формой (ФИО, email, роль, пароль) и отправкой данных в Users API.

features/change-language - инкапсулирует переключатель локали интерфейса (русский / английский) с синхронизацией vue-i18n и компонентов PrimeVue.

features/create-course-element - инкапсулирует модальное окно добавления и редактирования элемента темы курса (видео, файл, задание, тест) с загрузкой файла и переходом в конструктор quiz при выборе типа «тест».

features/import-excel - инкапсулирует массовый импорт данных из Excel: кнопку, модальное окно выбора файла и типа импорта (пользователи или оргструктура) с отправкой на соответствующий endpoint.

features/toggle-theme - инкапсулирует переключение светлой и тёмной темы оформления через глобальный стор темы.

features/create-question-bank - инкапсулирует модальное окно создания нового банка вопросов (название, описание) с передачей данных в API банков.

4.1.4. Слой widgets

Слой виджетов выполняет роль композиционного слоя для объединения сущностей и фич в самостоятельные, крупные блоки интерфейса страниц. Ниже приведены описания для некоторых из используемых виджетов приложения:

widgets/users-table - содержит таблицу пользователей админ-панели с поиском, фильтрацией по роли, пагинацией, удалением записи и переходом в профиль пользователя; включает стор загрузки и состояния таблицы.

widgets/teacher-profile-settings - содержит форму личных настроек преподавателя: редактирование email, смена пароля, загрузка аватара и блок назначенных ролей и прав доступа.

widgets/course-content-editor - содержит редактор содержания курса: список тем, элементы внутри тем (создание, редактирование, удаление, индикация прогресса) и интеграцию с модалками создания тем и элементов.

widgets/admin-sidebar - содержит боковую навигацию административного портала (пользователи, организация, профиль, выход) с подсветкой активного раздела.

widgets/admin-header - содержит верхнюю панель админ-портала с переключателями языка и темы и элементами служебного интерфейса (уведомления).

widgets/security-settings-card - содержит карточку настроек безопасности: форму смены пароля с валидацией и опциональный блок двухфакторной аутентификации; используется в профилях администратора и преподавателя.

widgets/assessment-question-map - содержит боковую карту вопросов при прохождении теста: сетку статусов (текущий, отвечен, не отвечен, помечен), индикатор прогресса и кнопку «Завершить и отправить» с диалогом подтверждения.

widgets/bank-questions-table - содержит таблицу вопросов выбранного банка с поиском, фильтром по типу вопроса, одиночным и массовым удалением и вызовом редактора вопроса.

4.1.5. Слой pages

Слой содержит компоненты экранов приложения. Страницы сопоставлены с путями во Vue Router и выполняют роль композиционной основы. Ниже приведены описания для некоторых из используемых страниц приложения:

pages/admin-profile - страница профиля администратора: карточка личных данных, настройки безопасности и сохранение изменений email и пароля.

pages/login - страница входа в систему в общем auth-layout с формой авторизации по email и паролю.

pages/organization - страница управления организационной структурой: дерево подразделений, карточка выбранного узла, создание и редактирование, импорт из Excel.

pages/student-courses - страница списка курсов студента с заголовком раздела и сеткой карточек активных и завершённых курсов.

pages/student-course-details - страница содержания выбранного курса: информация о курсе, прогресс, список тем и элементов с переходом к материалам и тестам.

pages/teacher-bank-questions - страница редактирования вопросов конкретного банка: таблица вопросов, создание и изменение через модальный редактор, удаление.

pages/teacher-courses - страница курсов преподавателя: сводная статистика, поиск, сетка карточек курсов и переход к созданию или деталям курса.

pages/teacher-quiz-builder - страница конструктора теста: настройки assessment (время, попытки, банк вопросов) и сохранение quiz как элемента курса.

pages/not-found - страница ошибки 404 с переключателями языка и темы и кнопками возврата на главную (с учётом роли) или назад по истории браузера.

4.1.6. Слой app

Верхний слой содержит код инициализации приложения. В директории `app/providers/router` настроена конфигурация путей Vue Router с глобальными навигационными хуками для контроля доступа. В директории `app/styles` задаются css токены и глобальные стили приложения. В файле `app/App.vue` находится корневой компонент vue приложения. В файле `app/main.ts` задается конфигурация vue приложения

Практическое применение описанной ранее архитектурной методологии FSD позволило гарантировать, что каждый модуль платформы имеет очерченные границы и единственную зону ответственности. Направление импортов исключительно сверху вниз исключило появление циклических зависимостей и сделало кодовую базу устойчивой к изменениям. Стала доступна возможность изменять детали реализации конкретной фичи на слое `features`, будучи уверенным, что эти изменения не затронут инфраструктурный слой `shared` или базовые сущности `entities`, что полностью удовлетворяет целям обеспечения масштабируемости и расширяемости разработанной платформы.

4.2. Проектирование и реализация ключевых сценариев

Проектирование пользовательского интерфейса образовательной платформы не сводится к отдельным экранным формам: каждый бизнес-процесс должен быть согласован с общей архитектурой приложения, правилами разграничения доступа и контрактами backend-сервисов. В данной главе анализируются ключевые сквозные сценарии, охватывающие основной жизненный цикл работы системы: аутентификация и маршрутизация по ролям; администрирование пользователей и иерархии университета; подготовка преподавателем банков вопросов и учебных курсов; прохождение студентом тестирования.

При описании реализации используется декомпозиция по FSD: страницы (pages) выступают точками сборки сценария, виджеты (widgets) инкапсулируют крупные блоки интерфейса, фичи (features) содержат законченные пользовательские действия, сущности (entities) - предметную область, API и хранилища данных. Такое разделение ответственности упрощает сопровождение системы и повторное использование компонентов в смежных сценариях (например, импорт Excel применяется и при добавлении пользователей, и при загрузке организационной структуры).

Для наглядности каждый сценарий сопровождается фрагментами описания интерфейса и диаграммами последовательности, отражающими обмен сообщениями между клиентскими модулями и серверными эндпоинтами.

4.2.1. Реализация системы аутентификации, авторизации и динамического ролевого разграничения доступа

Система поддерживает три роли пользователей: администратор, преподаватель и студент. Каждая роль после входа попадает в отдельный функциональный портал со своим меню и набором доступных страниц. Неавторизованный пользователь видит только экран входа; попытка открыть защищенный раздел без авторизации возвращает его на страницу логина с сохранением адреса для последующего перехода.

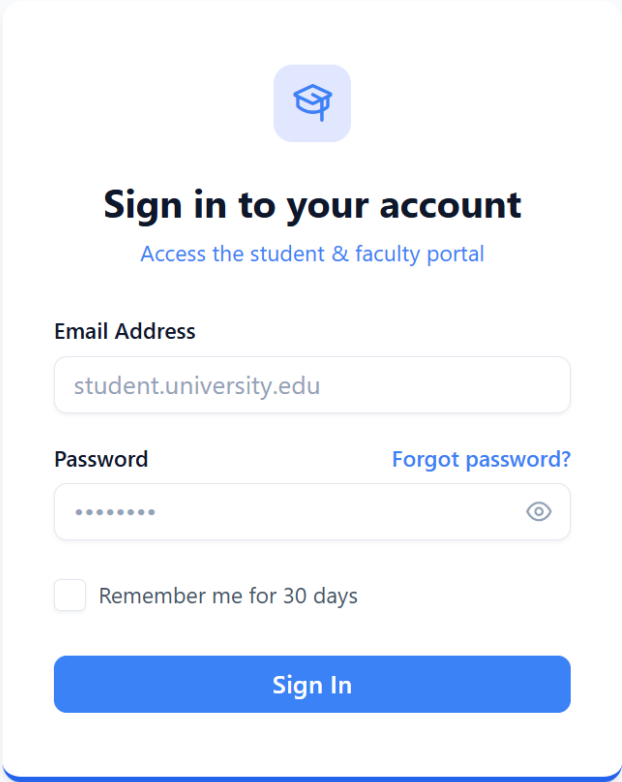
Интерфейс входа в систему

Страница авторизации (путь в приложении /login) оформлена в минималистичном макете без бокового меню: форма расположена по центру экрана на нейтральном фоне (рис. 12). Состав формы:

- логотип и название платформы;
- поле «Email / логин»;
- поле «Пароль» с возможностью показать/скрыть ввод;
- ссылка «Забыли пароль?»;
- флажок «Запомнить меня»;
- кнопка «Войти» с индикатором загрузки на время проверки учетных данных.

При успешном входе показывается приветственное уведомление, после чего пользователь автоматически перенаправляется в портал, соответствующий его роли. Если до входа он пытался открыть конкретную страницу, система возвращает его именно туда.

При ошибке (неверный логин или пароль) отображается сообщение об ошибке; форма остается на экране для повторной попытки.



The image displays a user authentication interface on a light blue background. At the top center is a blue square icon containing a white graduation cap. Below the icon, the text "Sign in to your account" is written in bold black font, followed by "Access the student & faculty portal" in a smaller blue font. The form contains two input fields: "Email Address" with the placeholder text "student.university.edu" and "Password" with a masked input (dots) and a "Forgot password?" link in blue. Below these fields is a checkbox labeled "Remember me for 30 days". At the bottom of the form is a large blue button with the text "Sign In".

Рис. 12. Интерфейс авторизации пользователя

Рольевые порталы после входа

После аутентификации интерфейс приложения динамически меняется в зависимости от роли. Каждый портал имеет общую структуру «боковое меню + рабочая область», но различается составом пунктов навигации и доступными разделами (рис. 13).

Портал администратора (путь в приложении /admin)

Предназначен для управления пользователями и организационной структурой университета.

Боковое меню:

- Пользователи;
- Структура (иерархия университета);
- Профиль администратора;
- Выход.

Стартовая страница - список пользователей с возможностью поиска, фильтрации и добавления новых учётных записей.

Портал преподавателя (путь в приложении /teacher)

Ориентирован на подготовку и ведение учебных курсов.

Боковое меню:

- Мои курсы;
- Банки вопросов;
- Профиль;
- Выход.

Меню может быть свёрнуто до иконок для экономии места на экране. Стартовая страница - список курсов преподавателя.

Портал студента (путь в приложении /student)

Содержит только функции, необходимые для обучения.

Боковое меню:

- Мои курсы;
- Профиль;
- Выход.

Стартовая страница - список курсов, на которые записан студент.

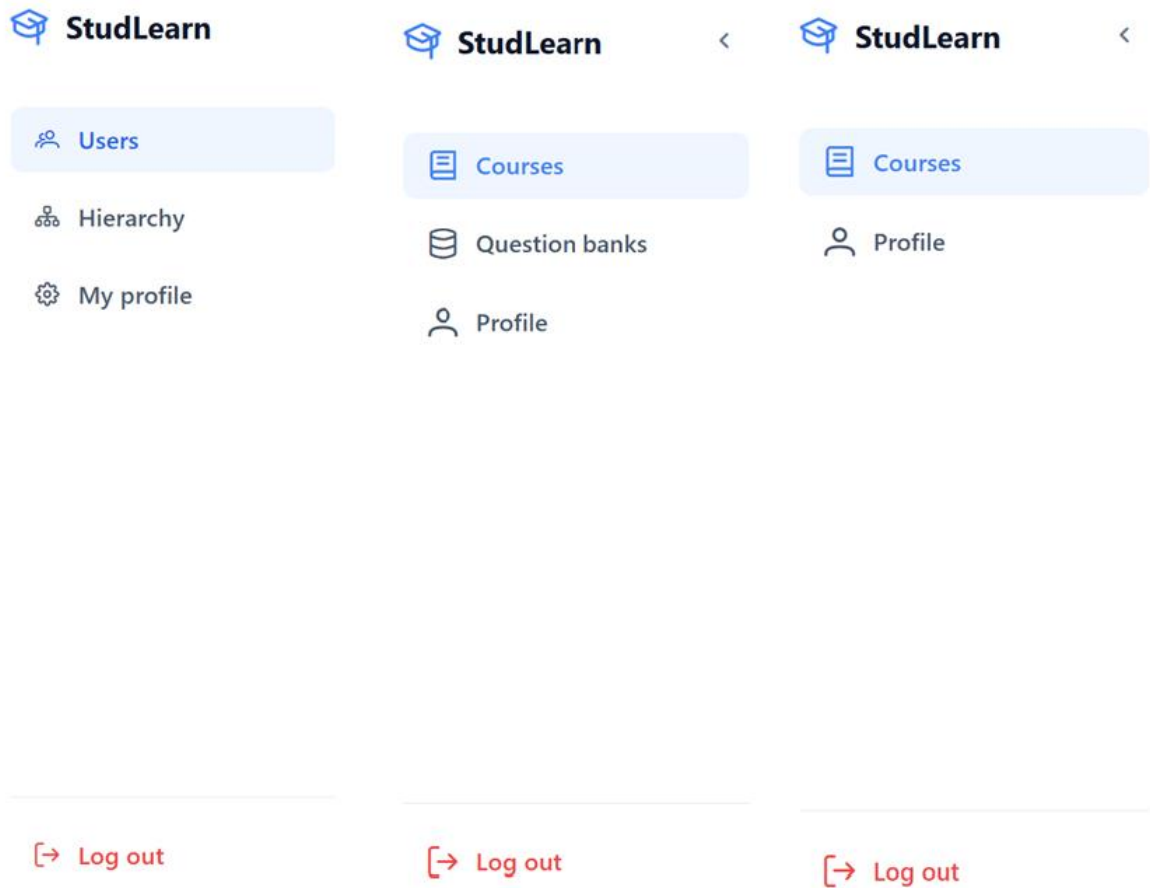


Рис. 13. Интерфейсы пользовательских боковых меню в зависимости от роли

Разграничение доступа

Разграничение реализовано на уровне маршрутизации: каждый URL привязан к одной или нескольким ролям. Поведение системы с точки зрения пользователя представлена в таблице 5: Таблица 5.

Описание маршрутизации внутри интерфейса после авторизации

<i>Ситуация</i>	<i>Результат</i>
<i>Гость открывает защищённую страницу</i>	<i>Переход на /login, после входа - возврат на запрошенную страницу</i>

<i>Авторизованный пользователь открывает /login</i>	<i>Автоматический переход в свой портал</i>
<i>Студент пытается открыть /admin или /teacher</i>	<i>Перенаправление на /student/courses</i>
<i>Преподаватель пытается открыть /admin</i>	<i>Перенаправление на /teacher/courses</i>
<i>Аналогично для других «чужих» разделов</i>	<i>Пользователь всегда попадает в свой портал</i>

Таким образом, студент не видит административных и преподавательских экранов даже при прямом вводе адреса в браузере, а преподаватель не получает доступ к управлению пользователями и структурой университета. Роль определяется при входе из данных учётной записи на основе декодирования JWT-токена [30] и сохраняется на время сессии; при каждом переходе между страницами выполняется проверка соответствия роли и запрашиваемого раздела (RBAC) [8, 9].

Завершение сессии

Выход из системы доступен из любого портала через кнопку «Выход» в нижней части бокового меню. После выхода пользователь возвращается на страницу авторизации, доступ к защищённым разделам блокируется.

При длительной неактивности или истечении срока действия сессии система также завершает авторизацию и перенаправляет на экран входа - пользователю не нужно вручную очищать данные браузера.

4.2.2. Клиентские компоненты интерфейса администратора

4.2.2.1. Реализация модуля администрирования пользователей (создание, импорт, редактирование ролей)

Модуль предназначен для централизованного управления учётными записями в портале администратора (путь в приложении /admin/users). Администратор может просматривать список пользователей, создавать записи по одной, импортировать группы из Excel, переходить к карточкам студентов и преподавателей для редактирования данных и прав доступа, а также удалять пользователей (рис. 14).

Обзорный экран «Пользователи»

Шапка страницы содержит заголовок, краткое описание раздела и две основные кнопки:

- «Импорт Excel» - массовая загрузка преподавателей или студентов;
- «Добавить пользователя» - поштучное создание учётной записи.

Ниже шапки расположена карточка статистики с общим числом пользователей в системе.

Основная область - таблица пользователей с инструментами фильтрации и поиска.

User	Role	Department	Status	Actions
Teacher Teacher Teacher admin@admin.ru	Teacher	АГДМ	Active	
Teacher Default teacher@example.com	Teacher	—	Active	
Андреев Андрей teacher@gmail.com	Teacher	МОСТ	Active	
Антипин Антон Александрович teacher@gmail.com	Teacher	МОСТ	Active	
Петров Иван Сергеевич 123@123.ru	Teacher	ДУМЧА	Active	
Темпов Темп asd@123.ru	Teacher	МОСТ	Active	
Юнин Данила 123 123@yandex.ru	Teacher	ДУМЧА	Inactive	

Рис. 14. Интерфейс модуля администрирования пользователей

Таблица пользователей

Над таблицей расположены:

- вкладки фильтра по роли - «Все», «Администратор», «Преподаватель», «Студент»;

- поле поиска - по имени.
- Колонки таблицы:
- Пользователь - аватар с инициалами, ФИО и email;
- Роль - цветная метка (Tag);
- Подразделение - кафедра или учебная группа;
- Статус - активен, неактивный.;
- Действия - кнопка удаления с подтверждением.

Таблица поддерживает постраничную навигацию. Клик по строке студента или преподавателя открывает карточку профиля для редактирования. Строки администраторов в таблице отображаются, но отдельный профиль для них не предусмотрен.

Создание пользователя

Поштучное создание выполняется через модальное окно с формой (рис. 15). Общие поля (для всех ролей):

- логин (username);
- email;
- фамилия, имя, отчество;
- роль - выпадающий список: администратор, преподаватель, студент.
- Дополнительные поля зависят от выбранной роли:
- для преподавателя - обязательный выбор кафедры;
- для студента - выбор учебной группы (и при необходимости кафедры).

При смене роли связанные поля сбрасываются, чтобы администратор не сохранил некорректную комбинацию. Обязательные поля проверяются до отправки; при ошибках поля подсвечиваются.

После успешного создания модальное окно закрывается, показывается уведомление, таблица обновляется.

Add new user [X]

Username
e.g. s102344

Email address
student.petrov.university.edu

Last name
e.g. Petrov

First name
e.g. Ivan

Middle name
e.g. Sergeevich

Role
Students

Department (optional)
Select a department

Student group
Select a group

[X] Cancel [✓] Save user

Рис. 15. Форма создания пользователя

Импорт пользователей из Excel

Массовое добавление доступно через кнопку «Импорт Excel» (рис. 16). Шаги в модальном окне:

- выбор типа импорта - преподаватели или студенты (роль задаётся типом файла);
- загрузка файла .xlsx / .xls - перетаскиванием или через диалог выбора;
- подтверждение импорта.

После обработки отображается число успешно импортированных записей, таблица пользователей обновляется.

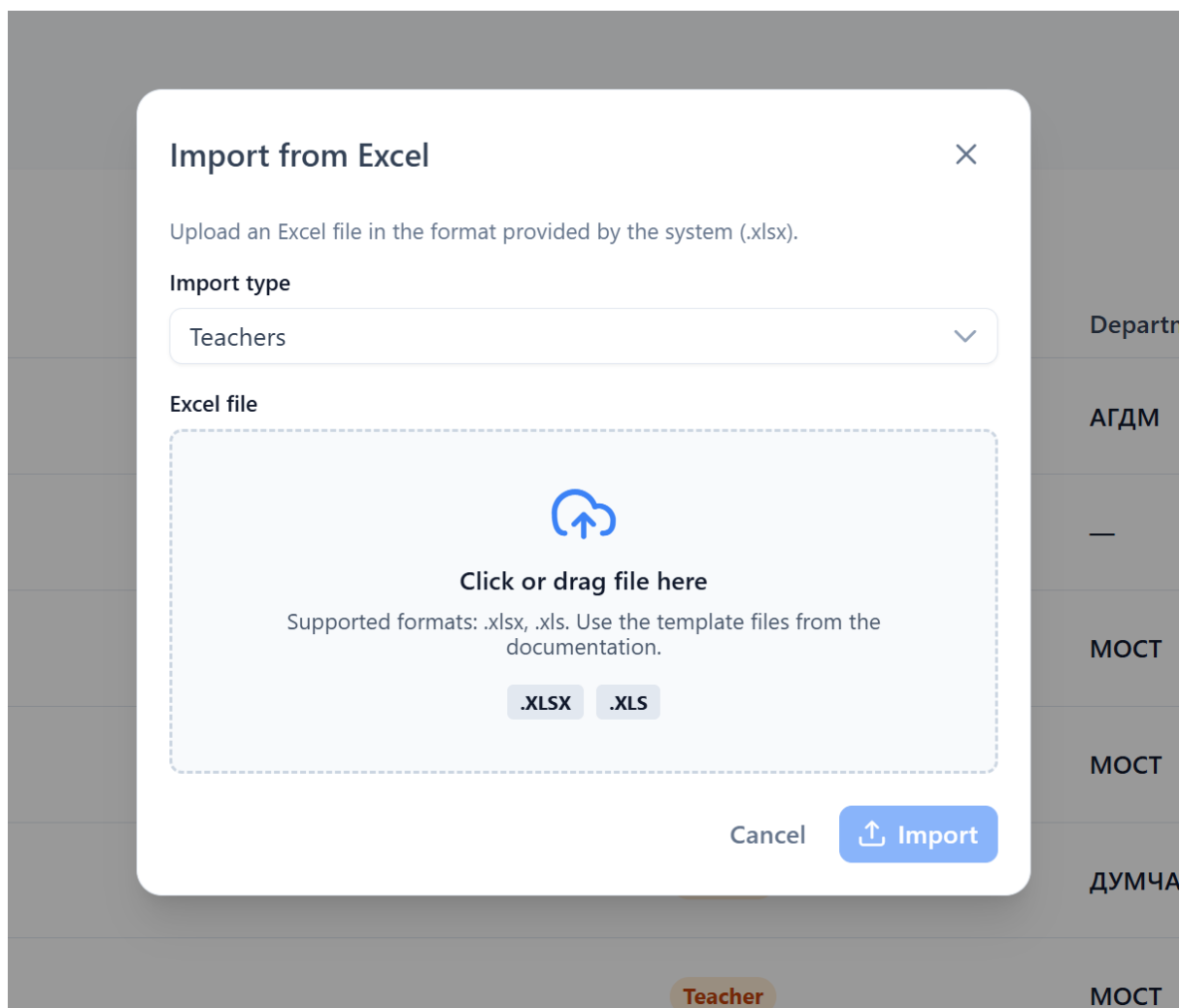


Рис. 16. Интерфейс массового импорта пользователей

Редактирование профиля студента

Переход из таблицы открывает страницу профиля студента с хлебными крошками (Пользователи → Студенты → ФИО).

Карточка «Информация о профиле» (ProfileInfoCard):

- аватар с инициалами;
- редактируемые: фамилия, имя, отчество, email;
- роль - отображается меткой, не редактируется (назначается при создании);
- read-only: табельный номер, институт, направление;
- редактируемый выбор учебной группы из справочника.

Карточка «Безопасность» - смена пароля пользователя администратором.

Внизу страницы - кнопки «Сбросить» и «Сохранить изменения» (активна только при наличии правок).

Редактирование профиля преподавателя

Страница профиля преподавателя имеет ту же навигационную структуру и заголовок «Права доступа» (рис. 17).

Карточка профиля - аналогична студенческой:

- редактирование ФИО и email;
- роль «Преподаватель» - только для просмотра;
- выбор кафедры из справочника.
- Карточка «Роли и права доступа» - управление областями доступа преподавателя к группам студентов:
- список назначенных областей в виде чипов (название · тип: факультет / направление / группа);
- удаление области по клику на крестик;
- кнопка «Добавить право» - модальное окно с каскадным выбором: тип области → факультет → направление → группа.

Карточка «Назначенные курсы» - просмотр курсов преподавателя (информативный блок).

Карточка «Безопасность» - смена пароля.

Таким образом, для преподавателя «редактирование ролей» в интерфейсе означает не смену системной роли, а настройку прав доступа к учебным подразделениям.

Permissions & account

Profile information

ИП

LAST NAME

Петров

FIRST NAME

Иван

PATRONYMIC

Сергеевич

ROLE

Teacher

EMAIL ADDRESS

1234@123.ru

EMPLOYEE ID

12344321

MAIN DEPARTMENT

ДУМЧА

Roles & permissions

Access levels and administrative groups assigned to this user

+ Add permission

ACCESS SCOPES

Институт информационных технологий математики и механики · Faculty

Assigned courses

View all history

Course code	Course name	Semester	Status

Security & access

Password reset

Generate new

Send link

Рис. 17. Интерфейс редактирования профиля преподавателя

Удаление пользователя

Удаление выполняется из таблицы: кнопка в колонке «Действия» → диалог подтверждения с именем пользователя → после подтверждения запись удаляется, таблица обновляется.

4.2.2.2. Проектирование интерфейса управления организационной структурой университета (кафедры, направления, группы)

Модуль предназначен для построения и сопровождения иерархии университета в портале администратора (путь в приложении /admin/organization). Интерфейс позволяет вести две связанные, но различные модели структуры: учебную (для привязки студентов и направлений подготовки) и административную (для кафедр). Управление выполняется через древовидное представление с панелью деталей выбранного подразделения.

Общая компоновка страницы

Шапка содержит заголовок «Структура» и краткое описание назначения раздела (рис. 18).

Переключатель вкладок задаёт режим работы:

- Учебная структура - факультет → направление подготовки → учебная группа;
- Административная структура - факультет → кафедра.

При смене вкладки дерево перезагружается, выбранный узел сбрасывается. Переключение блокируется на время редактирования, чтобы не потерять несохраненные изменения.

Рабочая область разделена на две панели:

- слева - дерево подразделений
- справа - карточка свойств выбранного узла.

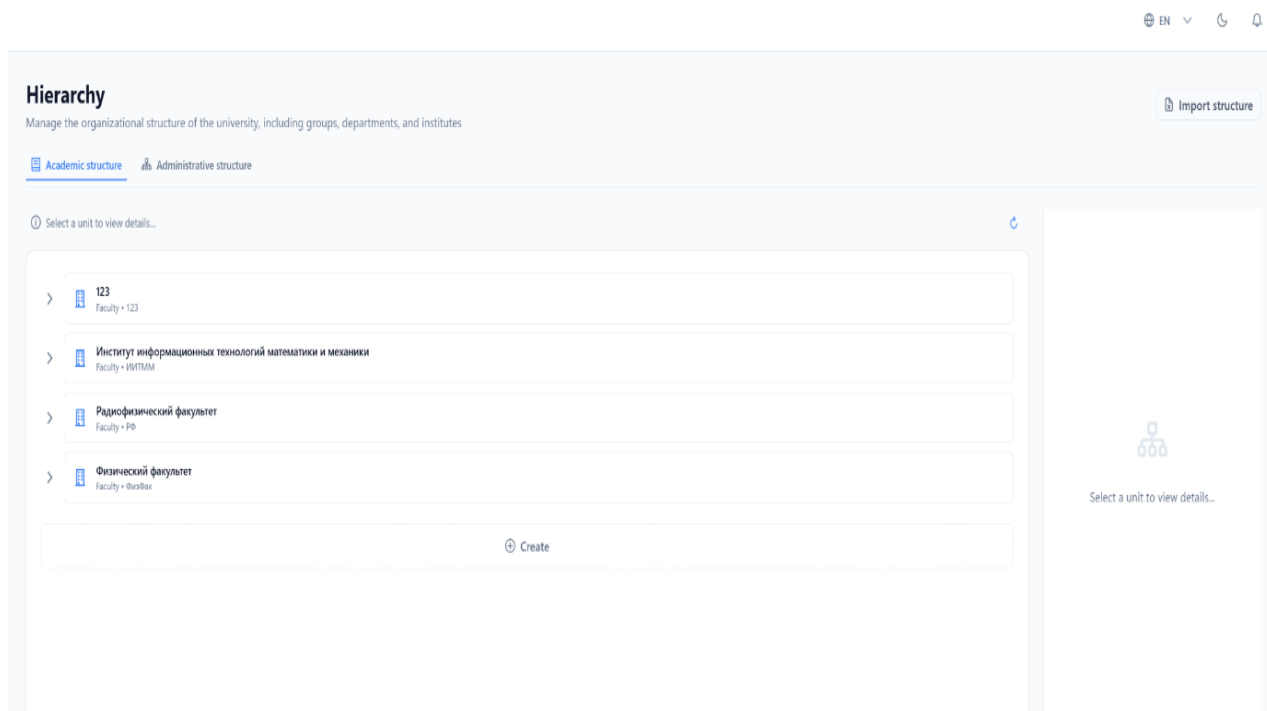


Рис. 18. Интерфейс управления организационной структурой университета

Древовидный навигатор (левая панель)

Панель инструментов над деревом:

- подсказка «Выберите подразделение для просмотра...»;
- кнопка обновления дерева - повторная загрузка с сервера.

Дерево (компонент Tree из библиотеки компонентов PrimeVue) отображает иерархию с ленивой подгрузкой: при первом открытии показываются факультеты; при раскрытии узла подгружаются дочерние элементы (направления, группы или кафедры - в зависимости от вкладки).

Каждый узел оформлен карточкой с:

- иконкой типа (факультет, направление, кафедра, группа);
- названием;
- подписью типа и краткого имени (для факультета).

Выбранный узел визуально выделяется. Во время редактирования в правой панели дерево получает полупрозрачный оверлей - выбор других узлов временно недоступен (рис. 19).

Кнопка «Создать» внизу дерева открывает модальное окно добавления подразделения.

Кнопка активна по правилам контекста:

- без выбранного узла - можно создать факультет (корень);
- на вкладке «Административная» - дочерний элемент можно добавить только к факультету (кафедра);
- на вкладке «Учебная» - к факультету (направление) или к направлению (учебная группа).

Если добавление недоступно, кнопка неактивна с подсказкой.

При первичной загрузке показываются скелетоны вместо дерева.

Панель деталей (правая колонка)

Если узел не выбран, отображается empty-state с иконкой и текстом-приглашением.

Если узел выбран, панель показывает:

- Режим просмотра:
 - иконка и тип подразделения (метка Tag);
 - полное название;
 - служебные поля: ID в базе, краткое имя (для факультета), при необходимости - ID родительского факультета или направления;
 - блок «Быстрые действия»:

- «Редактировать»;
- «Удалить подразделение» (с диалогом подтверждения).
 - Режим редактирования (по кнопке «Редактировать»):
 - поле названия;
 - для факультета дополнительно - поле краткого имени (код, напр. ENG);
 - кнопки «Сохранить изменения» и «Отмена».

Панель можно закрыть кнопкой «×» в углу - выбор узла сбрасывается.

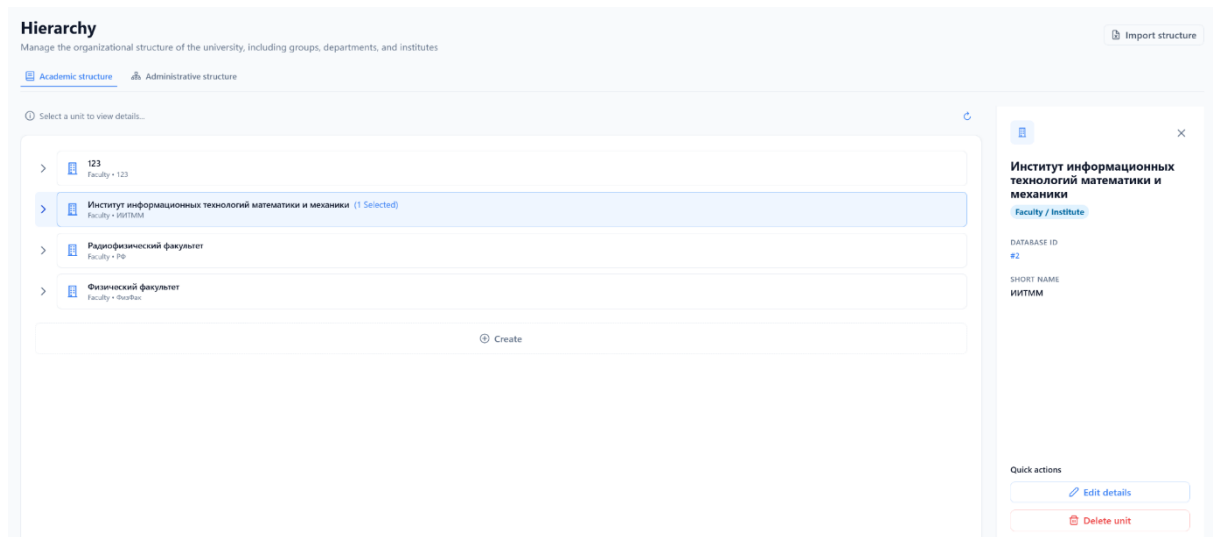


Рис. 19. Древовидное представление и панель свойств подразделения

Создание подразделения

Создание выполняется через модальное окно (рис. 20). Заголовок и поля формы зависят от контекста:

- если ничего не выбрано - создаётся факультет / институт;
- если выбран факультет на административной вкладке - кафедра;
- если выбран факультет на учебной вкладке - направление подготовки;
- если выбрано направление - учебная группа.
- В модальном окне отображаются:
 - родительский узел (название и иконка);
 - поле названия (обязательное);
 - для факультета - поле краткого имени (обязательное).

После успешного создания новый узел сразу появляется в дереве, родительский узел автоматически раскрывается, модальное окно закрывается, показывается уведомление об успехе.

Add new Faculty / Institute ✕

PARENT ROOT ORGANIZATION

Root organization

FACULTY / INSTITUTE NAME

Enter name...

This field is required

SHORT NAME

e.g. ENG

Cancel Create Faculty / Institute

Рис. 20. Форма добавления элемента организационной структуры

Таблица 6.

Две модели иерархии

<i>Вкладка</i>	<i>Уровни дерева</i>	<i>Назначение</i>
<i>Учебная структура</i>	<i>Факультет → Направление → Группа</i>	<i>привязка студентов к группам, отображение в профилях</i>

<i>Административная</i>	<i>Факультет → Кафедра</i>	<i>привязка преподавателей к кафедрам</i>
-------------------------	----------------------------	---

Обе модели используют один и тот же интерфейс (дерево + панель деталей), но загружают разные ветви иерархии. Это снижает когнитивную нагрузку: администратор работает в привычной схеме «список - детали», не переключаясь между разными экранами.

Связь с другими модулями

Справочники, сформированные в этом модуле, используются в модуле пользователей: при создании преподавателя выбирается кафедра, при создании студента - учебная группа. Корректное заполнение структуры университета - необходимое условие для последующего администрирования учётных записей.

4.2.3. Функциональный модуль преподавателя

4.2.3.1. Реализация конструктора курсов и управление структурой учебных материалов

Модуль предназначен для преподавателя и охватывает полный цикл работы с курсом: просмотр списка, создание, редактирование метаданных и построение структуры учебных материалов - от тем (разделов) до отдельных элементов контента (лекции, файлы, задания, тесты). Точка входа - раздел «Мои курсы» (путь в приложении /teacher/courses).

Список курсов

Шапка страницы содержит приветственный заголовок и описание раздела (рис. 21). Блок статистики - карточки с агрегированными показателями. Панель управления:

- поле поиска по названию и коду курса;
- кнопка «Добавить курс» - переход к форме создания.

Сетка карточек курсов: код, название, статус (черновик / активен), число студентов. Клик по карточке открывает страницу управления выбранным курсом.

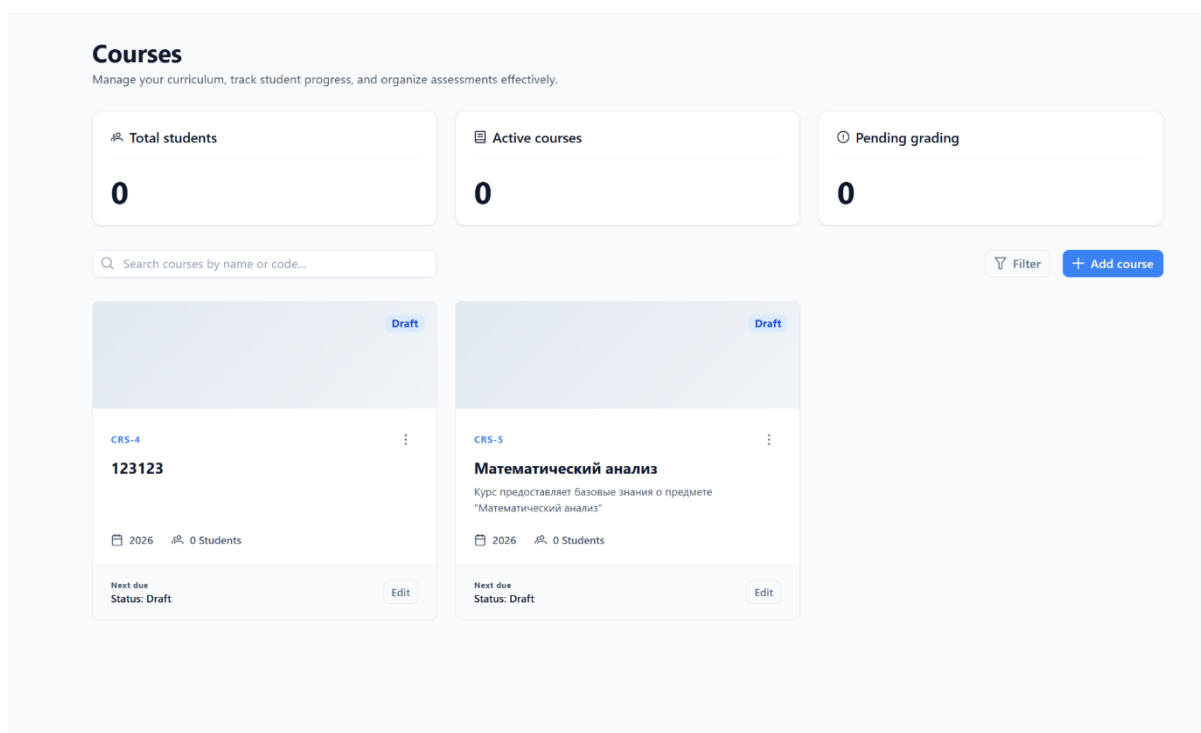


Рис. 21. Интерфейс списка курсов

Создание и редактирование сведений о курсе

Создание выполняется на отдельной странице (путь в приложении `/teacher/courses/new`) (рис. 22). Редактирование - на вкладке «Основная информация» внутри карточки курса. В обоих случаях используется виджет CourseInfoEditor.

Левая часть - форма:

- Название курса (обязательное);
- Описание - многострочное поле со счётчиком символов;
- Статус (только при редактировании) - «Черновик» или «Активен»;
- кнопки «Отмена» и «Создать курс» / «Сохранить изменения».
- Правая колонка - вспомогательная панель:
- предпросмотр карточки курса - как её увидит студент;
- блок «Советы преподавателю» с краткими рекомендациями по заполнению.

После создания курс открывается на вкладке основной информации; дальнейшее наполнение выполняется на вкладке «Содержание».

Create new course

Manage your curriculum, track student progress, and organize assessments effectively.

Course title *

e.g., Advanced Data Structures

Course description

Provide a detailed overview of what students will learn in this course...

0/500 characters

Cancel

Create course

COURSE CARD PREVIEW

Draft

CRS-NEW

New Course Title

Course description will appear here. It gives students a quick overview...

2026

0 Students

Next due

Status: Draft

Edit

Instructor tips

- Keep titles concise and descriptive.
- Use the description to outline learning objectives.
- Add course content after creation on the Content tab.

Рис. 22. Интерфейс создания учебного курса

Страница управления курсом

После выбора курса открывается оболочка с вкладками (TeacherCourseDetailsPage):

Таблица 7.

Вкладки в интерфейсе курса

Вкладка	Назначение
Основная информация	название, описание, статус
Содержание	структура тем и материалов
Участники	список студентов курса
Оценки	журнал успеваемости

Шапка страницы: кнопка «Назад к курсам», код и название курса, метка статуса. Раздел «Содержание» - центральный для конструктора учебных материалов.

Редактор содержания курса

Виджет CourseContentEditor построен в две колонки: основная область с темами и боковая панель со сводкой (рис. 23).

Панель инструментов

- «Добавить тему» - новый раздел курса;
- кнопки свернуть / развернуть все темы;
- «Обновить» - перезагрузка содержания с сервера.

Список тем

Каждая тема оформлена карточкой-аккордеоном:

Шапка темы (всегда видна):

- название и краткая метайнформация (число элементов, длительность);
- кнопки редактирования и удаления;
- chevron для сворачивания / разворачивания.
- Тело темы (при раскрытии):
- список элементов контента;
- в каждой строке: иконка типа, название, цветная метка, дополнительная информация (длительность, размер, статус);
- кнопки редактирования и удаления;
- кнопка «Добавить элемент в тему».

Внизу списка - зона «Добавить новую тему» с пояснением.

Таблица 8.

Доступные типы материалов в курсе

<i>Тип в интерфейсе</i>	<i>Метка</i>	<i>Назначение</i>
<i>Видеолекция</i>	<i>VIDEO</i>	<i>видеоматериал</i>
<i>Файл / reading</i>	<i>PDF</i>	<i>документ для самостоятельного изучения</i>
<i>Тест</i>	<i>TEST</i>	<i>контрольное задание</i>

Типы различаются цветом и иконкой, что упрощает навигацию по длинному содержанию курса.

Боковая панель редактора

- «Быстрое добавление» - сетка с типами материалов (видео, тест, файл);
- «Сводка по содержанию» - число тем, элементов, ориентировочная длительность;
- индикатор заполненности (progress bar) и подсказка по рекомендуемому составу курса;
- блок помощи со ссылкой на руководство.

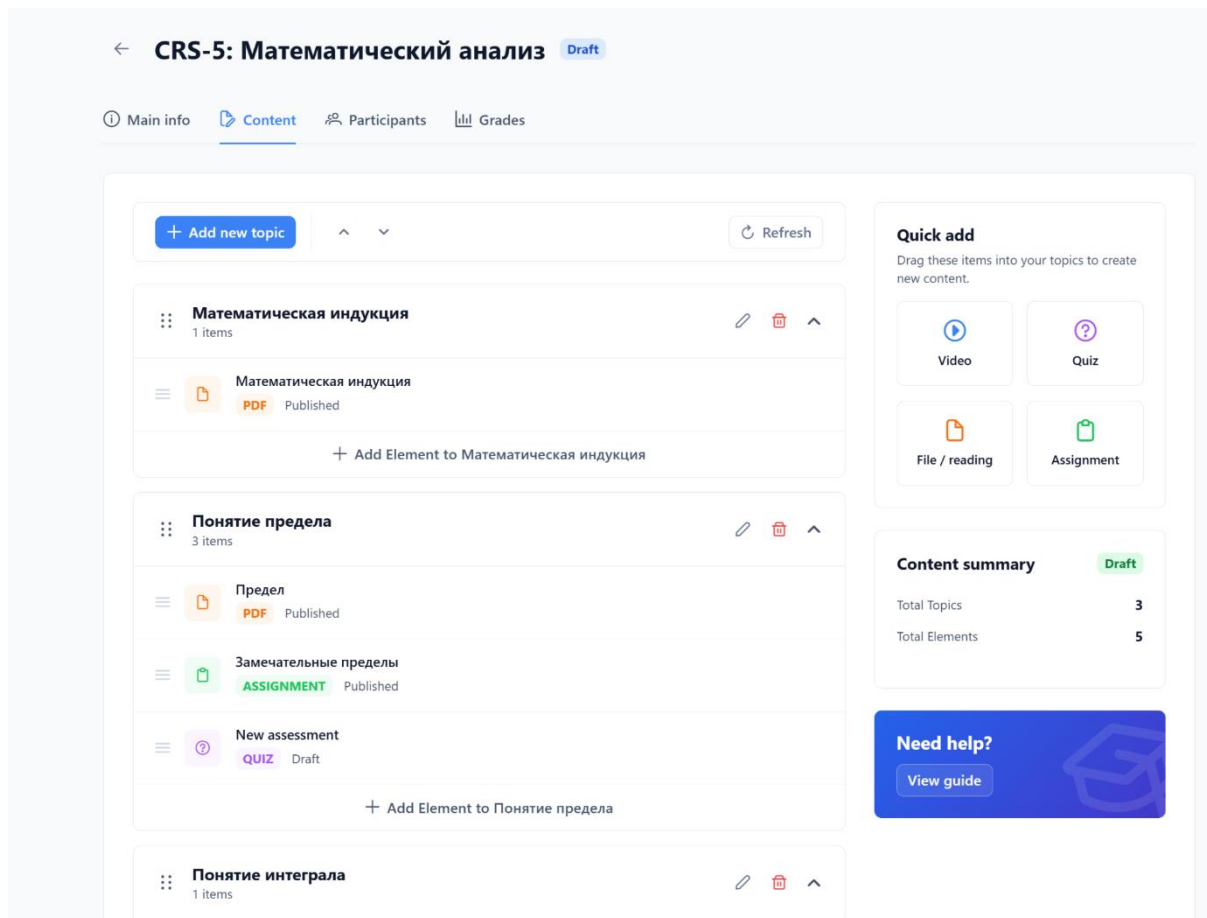


Рис. 23. Интерфейс управления структурой учебных материалов курса

Модальные окна добавления темы и материала

Создание / редактирование темы:

- название темы (обязательное);
- необязательное поле метаданных.

Создание / редактирование элемента:

- выбор типа контента из выпадающего списка;
- для видео и файлов - загрузчик файла с автоподстановкой названия и размера;
- поле названия (обязательное);
- поле дополнительной информации (длительность, срок сдачи и т.п.).

Для типа «Тест» модальное окно не сохраняет элемент локально, а перенаправляет в конструктор теста - отдельный экран сборки проверочного задания из банков вопросов. Редактирование существующего теста также открывает конструктор.

Логика работы преподавателя

Типичный сценарий наполнения курса:

1. Создать курс и заполнить основную информацию.
2. Перейти на вкладку «Содержание».
3. Добавить темы (модули, недели, разделы).
4. Внутри каждой темы добавить элементы нужного типа.
5. Для тестов - перейти в конструктор и связать материал с банками вопросов.
6. При необходимости изменить порядок, отредактировать или удалить темы и элементы.

Структура «курс → тема → элемент» соответствует иерархии на сервере (курс → секция → элемент секции) и понятна преподавателю без технических терминов.

4.2.3.2. Разработка интерфейса создания банка вопросов и формирования проверочных тестов

Подготовка проверочных материалов в системе разделена на два этапа. Сначала преподаватель создает банки вопросов - переиспользуемые наборы заданий с вариантами ответов и баллами. Затем из одного или нескольких банков формируется проверочный тест, который включается в содержание учебного курса.

Управление банками вопросов

Точка входа - страница «Банки вопросов» в разделе преподавателя (путь в приложении - /teacher/question-banks). Шапка страницы содержит заголовок, краткое описание раздела и две кнопки: «Импорт» и «Создать банк» (рис. 24). Основная область построена по принципу «список - детали» и разделена на две колонки:

- Левая колонка - список всех банков преподавателя. Каждый элемент показывает название, описание и число вопросов. Выбранный банк визуально выделяется.

- Правая колонка - работа с выбранным банком. Если банк не выбран, отображается приглашение создать новый.
- При выборе банка доступны:
- редактирование названия и описания;
- удаление банка (с подтверждением);
- превью первых нескольких вопросов с указанием типа и баллов;
- переход к полному списку вопросов.
- Создание банка выполняется через модальное окно с полями:
- «Название» (обязательное)
- «Описание».

После сохранения новый банк появляется в списке и автоматически выбирается для дальнейшей работы.

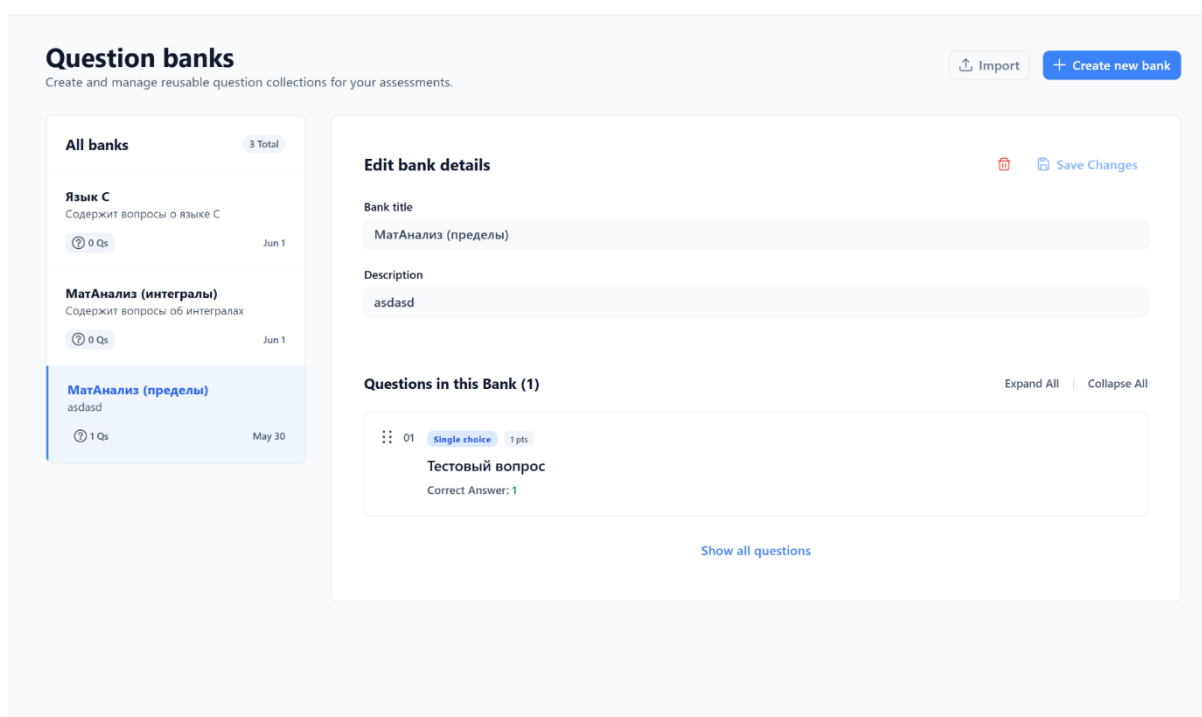


Рис. 24. Интерфейс управления банками вопросов

Наполнение банка вопросами

Полный список вопросов вынесен на отдельную страницу (путь в приложении /teacher/question-banks/:bankId), чтобы не перегружать экран обзора (рис. 25). Шапка содержит кнопку возврата, название банка, счётчик вопросов и кнопку «Добавить вопрос». Таблица вопросов включает:

- поиск по тексту;
- фильтр по типу (одиночный выбор, множественный выбор, текстовый ответ);
- колонки
 - номер
 - формулировка
 - тип
 - баллы
 - действия
- массовое удаление выбранных вопросов.
- Редактор вопроса открывается в модальном окне и содержит:
 - выбор типа вопроса;
 - поле формулировки;
 - поле «Баллы»;
 - для вопросов с выбором - список вариантов с отметкой правильных ответов (переключатель для одиночного выбора, флажки для множественного);
 - для текстовых вопросов - список допустимых эталонных ответов.

Типы вопросов визуально различаются цветными метками в таблице и в превью на главной странице банков.

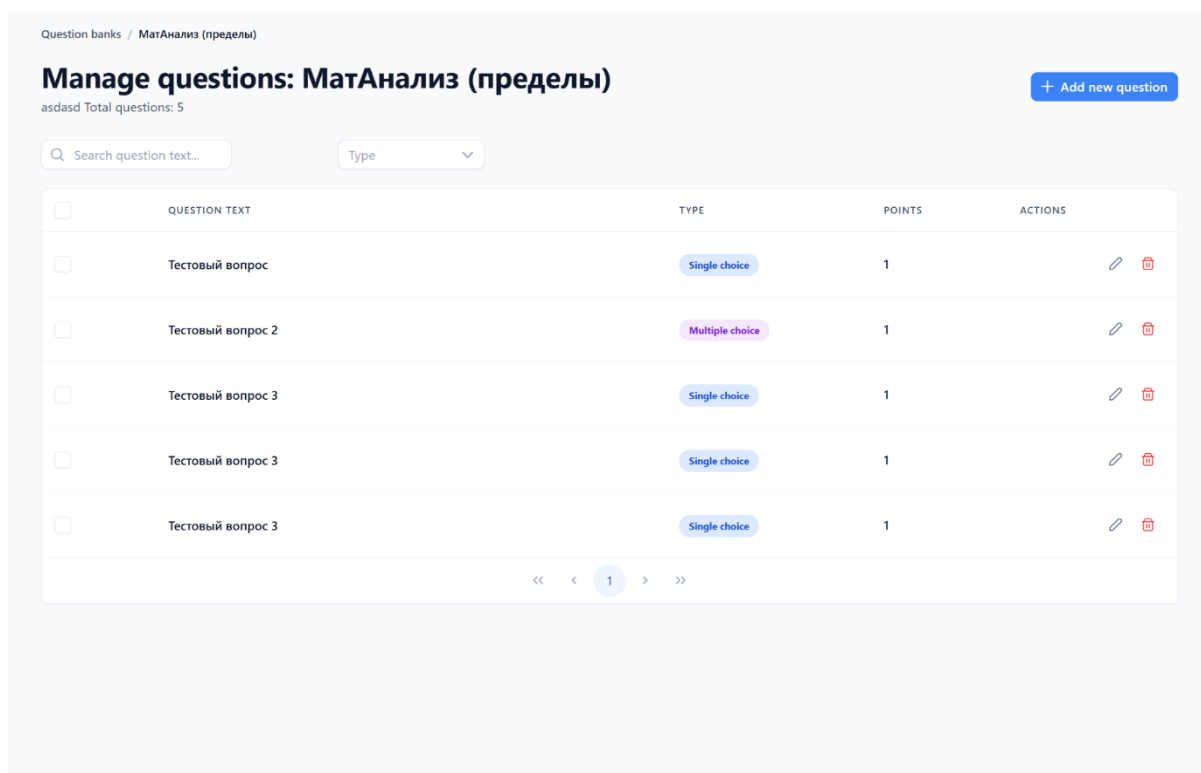


Рис. 25. Интерфейс редактирования вопросов в банке

Формирование проверочного теста

Тест создаётся в контексте курса - при добавлении элемента «Тест» в содержание или при редактировании существующего. Открывается страница конструктора теста (путь в приложении /teacher/courses/:id/quiz-builder) (рис. 26). Шапка содержит навигационную цепочку (Курсы → текущий курс → название теста), заголовок, индикатор статуса «Черновик» и кнопку сохранения. Основная область разделена на две колонки:

- Левая колонка - настройки теста в виде карточек:
 - Общая информация - название, инструкции для студента, срок сдачи, лимит времени, переключатели «Перемешивать вопросы» и «Показывать результаты».
 - Конструктор - список банков вопросов преподавателя. Для каждого банка можно:
 - включить или исключить его из теста;
 - указать, сколько вопросов случайно выбрать из банка;
 - задать балл за каждый вопрос из этого банка.

- Правая колонка - сводка: общее число вопросов в тесте, суммарный балл и кнопки «Сохранить черновик» / «Опубликовать».

Таким образом, преподаватель не дублирует вопросы в каждом тесте, а собирает контрольную работу из уже подготовленных банков, задавая правила выборки и оценивания.

Рис. 26. Интерфейс формирования проверочного теста из банков вопросов

4.2.4. Интерактивный интерфейс студента

4.2.4.1. Разработка интерфейса прохождения интерактивного тестирования (таймеры, валидация ответов)

Сценарий прохождения теста реализован в студенческом портале и состоит из двух последовательных экранов: информационной страницы перед началом и рабочей области непосредственного тестирования. Интерфейс спроектирован так, чтобы студент заранее видел условия контроля, а во время работы - контролировал время, прогресс и сохранность ответов.

Вход в тестирование

Студент открывает тест из содержания курса (элемент типа «Тест»). Сначала отображается информационная страница (рис. 27).

Навигационная цепочка: Курсы → код курса → название теста.

Шапка содержит название и краткое описание теста.

Карточки параметров:

- лимит времени (минуты);
- число заданий;
- срок сдачи.

Блок инструкций - нумерованный список правил прохождения (порядок действий, ограничения, поведение при истечении времени).

Кнопки действий:

- «Начать тест» - переход к экрану прохождения;
- «Вернуться к курсу» - отмена без начала сессии.

Под кнопками - примечание о согласии с правилами академической честности.

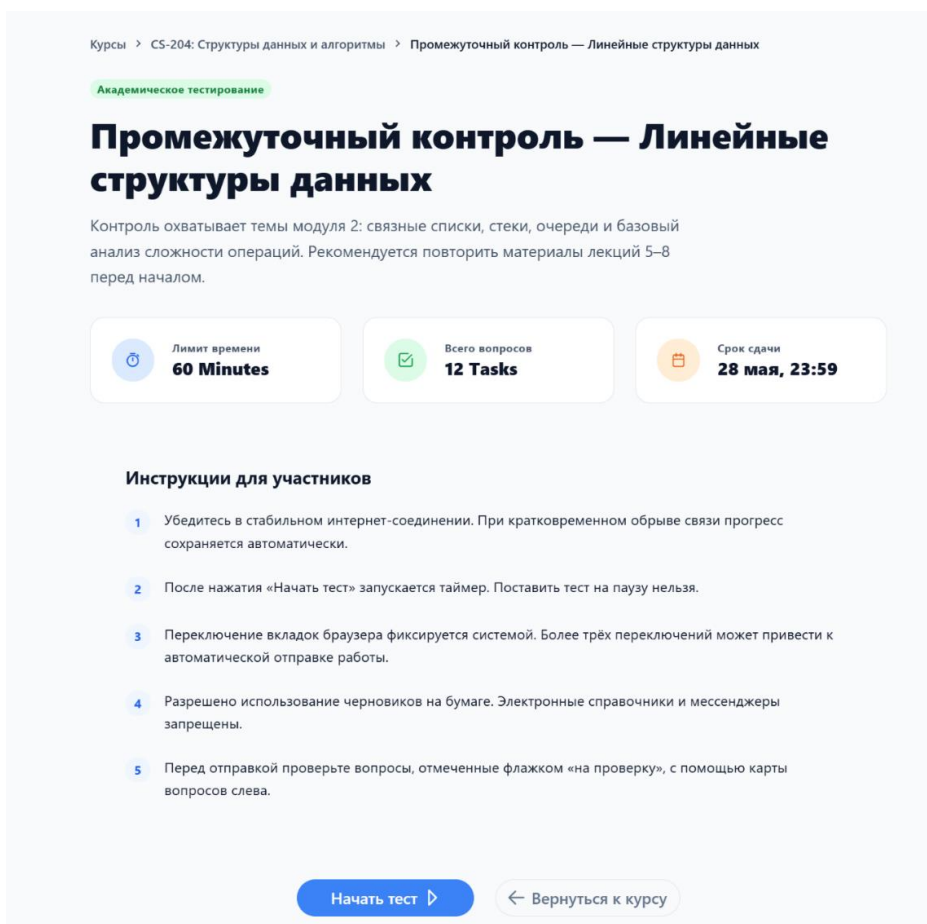


Рис. 27. Интерфейс подготовки к прохождению интерактивного тестирования

Рабочая область прохождения теста

После нажатия «Начать тест» открывается страница сессии (рис. 28). Макет разделен на шапку и двухколоночную рабочую зону.

Шапка сессии

- слева - название теста и название курса;
- справа - таймер обратного отсчёта в формате MM:SS.

Таймер постоянно виден на экране. Когда остаётся менее 5 минут, блок таймера меняет оформление (предупреждающий стиль), привлекая внимание студента к приближению дедлайна.

Левая колонка - карта вопросов:

Компактная панель навигации по всему тесту:

Сетка номеров вопросов - каждая ячейка показывает порядковый номер и визуальный статус:

- текущий вопрос - выделен;
- отвеченный - отмечен как заполненный;
- неотвеченный - нейтральное состояние.

На ячейке может отображаться флажок - вопрос помечен «на проверку» перед сдачей.

Индикатор прогресса - progress bar и процент отвеченных вопросов.

Действия:

- «Пометить для проверки» / «Снять пометку» - для текущего вопроса;
- «Завершить и отправить» - финальное завершение теста.

Клик по номеру вопроса мгновенно переключает рабочую область на выбранное задание.

Правая колонка - область ответа:

Отображается один вопрос за раз.

Заголовок вопроса:

- метка типа (например, «Multiple Choice», «Essay Question»);
- балл за вопрос;
- текст формулировки;

Область ввода ответа зависит от одного из типов, представленных в таблице 8:

Таблица 9.

Таблица сопоставления типов вопросов и компонентов интерфейса

<i>Тип</i>	<i>Элементы интерфейса</i>
<i>Одиночный выбор</i>	<i>карточки вариантов с <code><RadioButton /></code>; выбор кликом по карточке или переключателю</i>
<i>Развёрнутый ответ</i>	<i>многострочное поле <code><Textarea /></code>, панель форматирования, счётчики слов и символов</i>

Нижняя панель навигации:

- «Предыдущий вопрос» (неактивна на первом);
- точечный индикатор позиции в тесте;
- «Следующий вопрос» (неактивна на последнем).

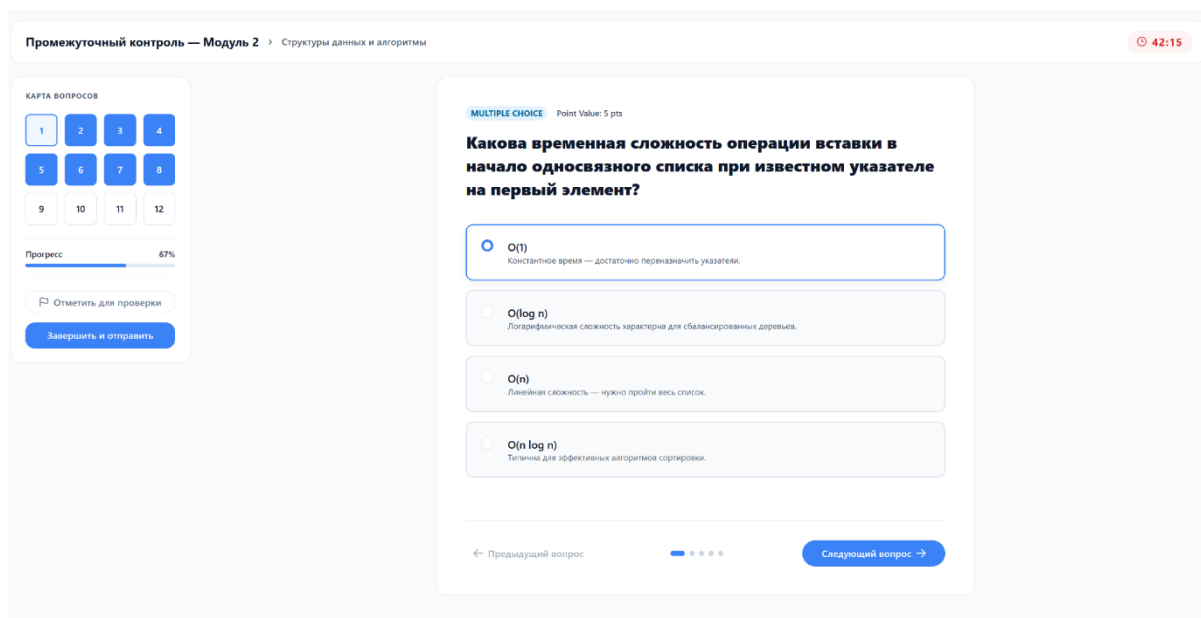


Рис. 28. Интерфейс прохождения интерактивного теста

Таймер и контроль времени

Таймер запускается при загрузке сессии и каждую секунду уменьшает оставшееся время. Формат отображения понятен студенту (42:15 - 42 минуты 15 секунд).

Визуальное предупреждение при малом остатке времени помогает принять решение о завершении или проверке неотвеченных вопросов. Таймер останавливается при выходе со страницы или после подтвержденной отправки теста.

Валидация и сохранение ответов

Валидация в интерфейсе реализована на нескольких уровнях - без блокирующих сообщений на каждый шаг, но с постоянной обратной связью.

1. Автосохранение ответа

При изменении ответа (выбор варианта или ввод текста) значение сразу отображается в интерфейсе и отправляется на сервер. Студент видит статус «черновик сохранён» и не теряет прогресс при переключении между вопросами.

2. Визуальный контроль полноты

Карта вопросов и progress bar показывают, сколько заданий уже отвечено. Неотвеченные вопросы остаются визуально отличимыми - студент может вернуться к ним до сдачи.

3. Подтверждение завершения

При нажатии «Завершить и отправить» открывается диалог подтверждения с разным содержанием:

- если все вопросы отвечены - нейтральное предупреждение о невозможности изменить ответы после сдачи;
- если есть пропуски - акцентное предупреждение с указанием, что часть вопросов осталась без ответа.

Студент может вернуться к тесту или подтвердить отправку. Кнопка подтверждения меняет акцент в зависимости от полноты ответов. После отправки показывается уведомление об успехе, выполняется возврат на страницу курса.

5. Тестирование приложения и автоматизация процессов разработки

Разработка клиентской части образовательной платформы сопровождается комплексом мер, обеспечивающих качество программного кода на протяжении всего жизненного цикла продукта. В текущей главе описывается стратегии тестирования и автоматизации, применение каждого инструмента и приводятся конкретные результаты, полученные в рамках разработки платформы.

5.1. Организация автоматических проверок качества кода с использованием Git-хуков

5.1.1. Назначение и место в процессе разработки

Первой ступенью по повышению качества кода в проекте является статический анализ - автоматическая проверка исходного кода без его выполнения. Статический анализ выявляет синтаксические ошибки, нарушения принятого стиля оформления и потенциально ошибочные конструкции на этапе редактирования или перед фиксацией изменений в репозитории.

В отличие от юнит-тестов, инструменты статического анализа работают с исходным текстом и не требуют запуска. Это делает их принципиально более быстрыми: полный цикл проверки занимает секунды, а не минуты. Ранее обнаруженная проблема обходится значительно дешевле, чем дефект, найденный на стадии тестирования.

В рамках проекта статический анализ реализован следующими инструментами:

- **ESLint** с плагинами для Vue 3 и TypeScript - анализирует семантику кода и специфические антипаттерны Vue-компонентов;
- **Prettier** - форматтер, обеспечивающий единообразное оформление кода вне зависимости от среды разработки конкретного участника команды.

Чтобы гарантировать запуск проверок на каждое изменение кодовой базы, используется механизм **Git-хуков**, автоматизированный через инструмент **Husky**.

5.1.2. Git-хуки, Husky и lint-staged

Git-хук - скрипт, который Git автоматически вызывает в ответ на определённые события: создание коммита, отправку изменений на сервер и другие. Если скрипт завершается с ненулевым кодом, соответствующее Git-действие прерывается.

Традиционная проблема хуков - их нахождение внутри директории `.git/`, которая не отслеживается системой контроля версий. **Husky** решает эту проблему: инструмент перенаправляет хуки в директорию `husky/`, которая фиксируется в репозитории наравне с остальным кодом. При выполнении `npm install` скрипт `prepare` автоматически инициализирует Husky, после чего хуки становятся доступны всем участникам без дополнительных действий.

В проекте настроены два хука. **.husky/pre-commit** делегирует работу инструменту `lint-staged`, который запускает проверки только для файлов, добавленных в индекс Git (`staged files`) - это исключает прогон всей кодовой базы при каждом коммите. **.husky/pre-push** выполняет полный прогон модульных тестов командой `npm run test:run` перед отправкой ветки в удалённый репозиторий.

Для Vue- и TypeScript-файлов применяется последовательная цепочка: ESLint проводит семантический анализ с учётом специфики Vue 3, Prettier финально форматирует результат. Порядок инструментов принципиален: ESLint исправляет логику до форматирования.

ESLint настроен в формате Flat Config. Слой `skipFormatting` (из `eslint-config-prettier`) делегирует всё форматирование Prettier.

5.1.3. Описание результатов

Интеграция Husky, lint-staged и линтеров обеспечивает следующие измеримые эффекты:

- **Автоматическое исправление** распространенных нарушений (`--fix`) снижает ручную работу разработчика по приведению кода к стандарту.
- **Блокировка коммита** при наличии неисправимых ошибок ESLint исключает попадание некорректного кода в историю репозитория.
- **Единый стиль** кодовой базы гарантируется Prettier - форматирование становится детерминированным вне зависимости от настроек IDE.
- **Быстрая обратная связь**: среднее время выполнения `pre-commit` хука составляет 3–4 секунды для типичного коммита из 3–5 изменённых файлов.

На (рис. 29) представлена схема прохождения изменений через все уровни автоматических проверок.

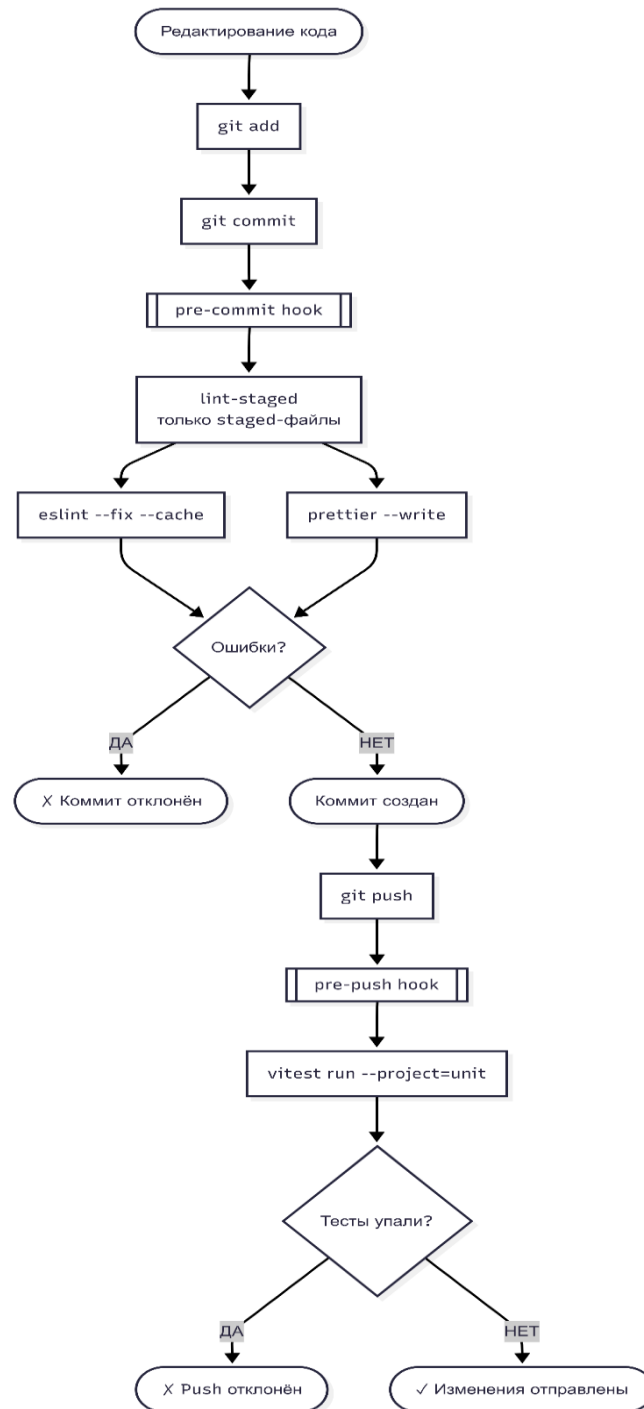


Рис. 29. Схема обработки кода при помощи автоматических проверок

5.2. Модульное и интеграционное тестирование бизнес-логики с использованием Vitest

5.2.1. Назначение и выбор инструмента

Второй уровень тестовой пирамиды занимают **модульные (unit) тесты** - небольшие проверки изолированных фрагментов кода: отдельных функций, сервисов, Zod-схем и компонентов. В отличие от E2E-тестов, которые проверяют пользовательский сценарий целиком, модульные тесты локализуют источник проблемы: падение конкретного теста немедленно указывает на конкретную функцию или ветку логики.

Для запуска тестов выбран **Vitest** [21] - тест-раннер, построенный на базе Vite (подробная информация дана в п. 3.6.1).

5.2.2. Конфигурация и структура тестов

Vitest использует концепцию **projects** - логически изолированных конфигураций внутри одного vitest.config.ts. В проекте определено два проекта: unit для модульных тестов и storybook для interaction-тестов Storybook (рассматривается в п. 5.3). Ключевые параметры проекта unit: наименование, окружение, файл настройки, определение регулярного выражения для тестовых файлов, настройки очистки и восстановления mock данных

Happy-DOM выбран как тестовая среда - реализация DOM API на чистом JavaScript, значительно быстрее аналога - JSDOM. Глобальный файл src/test/setup.ts сбрасывает localStorage перед каждым тестом и после него, обеспечивая изоляцию состояния между тест-кейсами.

Ниже в таблице 10 приведена информация по файлам, в которых описаны тесты.
Таблица 10.

Классификация тест-файлов проекта

Уровень	Файл	Тестируемая единица	Кол-во кейсов
Утилиты / хелперы	shared/lib/jwt.spec.ts	Хелперы истечения JWT-токена	3
Сервисы	shared/api/token.service.spec.ts	tokenService (get/set/clear в localStorage)	4

Интеграция	shared/api/auth-interceptor.spec.ts	Логика обновления access-токена	3
Маппер DTO	entities/user/lib/mappers.spec.ts	mapAdminUserListItemToUser	6
Маппер DTO	entities/organization/lib/mappers.spec.ts	Маппер организационных единиц	4
Builder	entities/organization/lib/buildPayload.spec.ts	Формирование payload оргструктуры	4
Бизнес-логика	features/add-org-unit/lib/resolveChildType.spec.ts	Определение типа дочернего узла дерева	7
Builder	features/add-user/lib/buildPayload.spec.ts	buildCreateAdminUserPayload	5
Схема валидации	features/add-user/lib/schema.spec.ts	Zod-схема формы добавления пользователя	5
Схема валидации	features/admin-user-profile/lib/schema.spec.ts	Схема формы редактирования профиля	3
Схема валидации	features/own-profile/lib/schema.spec.ts	Схема формы своего профиля	12
Компонент	features/add-user/ui/AddUserForm.spec.ts	Vue-компонент формы добавления пользователя	6
Builder	features/admin-user-profile/lib/build-update-payload.spec.ts	Формирование payload обновления профиля	5

5.2.3. Тесты чистых функций и хелперов

Наиболее изолированный уровень - тесты чистых функций без побочных эффектов. JWT-токен в проекте хранит поле exp - время истечения в формате Unix timestamp. Функции isAccessTokenExpired и isAccessTokenExpiringSoon используются в перехватчике запросов для принятия решения о необходимости обновления токена.

В тестах применяется вспомогательная фабрика `createTokenWithExp`, которая конструирует минимальный JWT с заданным `exp`. Это позволяет тестировать временную логику детерминировано без зависимости от реального времени

5.2.4. Интеграционные тесты

Файл `auth-interceptor.spec.ts` демонстрирует интеграционный уровень тестирования - проверку взаимодействия нескольких модулей: функции `refreshAccessToken`, сервиса `tokenService` и обработчика истёкшей сессии.

Тест заменяет глобальный `fetch` на мок с помощью `vi.stubGlobal`, избегая реальных HTTP-запросов (рис. 30). Особого внимания заслуживает тест дедупликации параллельных запросов на обновление токена: несколько запросов могут одновременно получить ответ 401. Тест гарантирует, что логика `pending-промиса` корректно ограничивает количество вызовов `/auth/refresh` до одного:

```
it('deduplicates concurrent refresh requests', async () => {
  const [first, second] = await Promise.all([refreshAccessToken(), refreshAccessToken()])

  expect(first).toBe('new-access-token')
  expect(second).toBe('new-access-token')
  expect(fetch).toHaveBeenCalledTimes(1)
})
```

Рис. 30. Пример интеграционного теста

5.2.5. Unit тесты функций форм

В проекте используется паттерн **фабрики тестовых данных** - функция `createDto` возвращает объект со значениями по умолчанию и предоставляет возможность переопределить нужные поля. Такой подход делает тесты декларативными и снижает объём кода при добавлении новых полей в DTO - изменение вносится только в фабрику.

Тест приоритизации ролей проверяет инвариант, важный для отображения интерфейса: если пользователь имеет несколько ролей одновременно, маппер должен вернуть наивысшую (рис. 31):

```

it('resolves primary role by priority when multiple roles are present', () => {
  const teacherUser = mapAdminUserListItemToUser(
    createDto({ roles: [ROLES.STUDENT, ROLES.TEACHER] })
  )
  const adminUser = mapAdminUserListItemToUser(
    createDto({ roles: [ROLES.STUDENT, ROLES.TEACHER, ROLES.ADMIN] })
  )

  expect(teacherUser.role).toBe('Teacher')
  expect(adminUser.role).toBe('Super Admin')
})

```

Рис. 31. Пример unit теста

Zod-схемы задают контракт для данных формы. Тесты схем верифицируют контракт через метод `safeParse()`. Пример теста условно-обязательного поля (рис. 32)

```

it('requires department for teacher', () => {
  const result = schema.safeParse({
    ...validBase,
    role: ROLES.TEACHER,
    departmentId: null,
    groupId: null
  })

  expect(result.success).toBe(false)
  if (!result.success) {
    expect(result.error.issues.some((issue) => issue.path[0] === 'departmentId')).toBe(true)
  }
})

```

Рис. 32. Пример unit теста на zod схему

Поскольку схемы - чистые объекты Zod без зависимости от Vue или DOM, тест-кейс выполняется за 2–5 мс, что позволяет покрывать большое число граничных случаев без значимых потерь во времени.

5.2.6. Компонентные тесты

Монтирование компонентов в тестах требует обеспечить тот же набор глобальных провайдеров, что и в реальном приложении: `vue-i18n`, `VueQueryPlugin` и `PrimeVue`. Для этого в проекте создана вспомогательная функция `renderWithApp` (рис. 34).

```

export const renderWithApp = <C extends Component>({
  component: C,
  options: MountingOptions<InstanceType<C>> = {}
}): VueWrapper<InstanceType<C>> => {
  const queryClient = createTestQueryClient()

  return mount(component, {
    ...options,
    global: {
      ...options.global,
      plugins: [
        i18n,
        [VueQueryPlugin, { queryClient }],
        [
          PrimeVue,
          {
            theme: primeVueThemeConfig
          }
        ],
        ...(options.global?.plugins ?? [])
      ]
    }
  })
}

```

Рис. 34. Реализация вспомогательной функции renderWithApp

QueryClient создаётся с отключенными повторными запросами (retry: false) и нулевым временем кэширования (gcTime: 0). Компонентный тест AddUserForm.spec.ts мокирует API-зависимости через vi.mock() и использует flushPromises() из @vue/test-utils [22] для ожидания разрешения всех промисов после монтирования:

```

it('hides department and group fields for admin role', async () => {
  const wrapper = mountForm()
  await flushPromises()

  await setSelectValue(wrapper, '#role', ROLES.ADMIN)
  await flushPromises()

  expect(wrapper.find('#group').exists()).toBe(false)
  expect(wrapper.find('#department').exists()).toBe(false)
})

```

Рис. 35. Пример компонентного теста

5.2.7. Покрытие кода

Измерение покрытия выполняется инструментом **V8 Coverage** (@vitest/coverage-v8). Командой `npm run test:coverage` формируются три вида отчётов: консольная таблица (text), интерактивный HTML-отчёт и машиночитаемый json-summary для CI-систем.

Таблица 11.

Результаты покрытия кода тестами (фрагмент вывода `npm run test:coverage`)

Модуль	Statements	Branches	Functions	Lines
shared/lib/jwt.ts	100 %	100 %	100 %	100 %
shared/api/token.service.ts	100 %	100 %	100 %	100 %
shared/api/auth-interceptor.ts	87 %	80 %	100 %	87 %
entities/user/lib/mappers.ts	95 %	92 %	100 %	95 %
features/add-user/lib/buildPayload.ts	100 %	100 %	100 %	100 %
features/add-user/lib/schema.ts	100 %	-	100 %	100 %
features/add-org-unit/lib/resolveChildType.ts	100 %	100 %	100 %	100 %

В конфигурации установлены минимальные пороговые значения (thresholds) 30% по всем метрикам. Это намеренно консервативная точка старта: пороги фиксируют регрессии (добавление кода без тестов), не требуя стопроцентного покрытия вспомогательных утилит и конфигурационных файлов.

5.3. Документирование и изолированная разработка компонентов в среде Storybook

5.3.1. Назначение и концепция

Разработка UI-компонентов в контексте приложения сопряжена с ограничениями: чтобы увидеть компонент в определённом состоянии, необходимо воспроизвести его через реальные взаимодействия - авторизоваться, создать объект, навигировать к нужной странице. Это замедляет итерации при работе над внешним видом и поведением компонентов.

Storybook решает эту проблем. Инструмент предоставляет изолированную среду, в которой каждый компонент отображается независимо с произвольно заданными входными данными. Каждое такое представление называется **story** - именованное зафиксированное состояние компонента. Набор stories служит одновременно живой документацией, средой тестирования и инструментом демонстрации.

В проекте используется связка **@storybook/vue3-vite** - официальный плагин Storybook для Vue 3 с интеграцией через Vite, что обеспечивает использование общего vite.config.ts и алиасов без дополнительной настройки.

5.3.2. Конфигурация и среда выполнения

Файл .storybook/preview.ts инициализирует глобальные провайдеры через setup() - аналог app.use() в main.ts реального приложения:

```

setup((app) => {
  app.use(createPinia())
  app.use(storybookRouter)
  app.use(i18n)
  app.use(VueQueryPlugin)
  app.use(PrimeVue, {
    theme: primeVueThemeConfig
  })
  app.use	ToastService)
  app.use(ConfirmationService)
})

```

Рис. 36. Инициализация Storybook

Vue Router инициализируется с `createMemoryHistory` вместо `createWebHistory` - история маршрутов хранится в памяти, что корректно работает в изолированной среде Storybook. Дополнительно подключаются глобальные стили: CSS reset и файл `tokens.css` с дизайн-токенами, без которых компоненты отображались бы без тематического оформления.

На уровне всех stories применяется декоратор, воспроизводящий фоновый цвет и типографику реального приложения:

5.3.3. Структура stories в FSD

В проект добавлены 20 файлов stories, размещённых в соответствующих FSD-слоях рядом с компонентами. Такое расположение следует принципу `co-location` - при переименовании или удалении компонента story-файл перемещается вместе с ним.

Таблица 12.

Файлы stories и соответствующие FSD-слои

Файл	FSD-слой	Компонент	Количество историй
shared/ui/widget-card/WidgetCard.stories.ts	shared	WidgetCard	3
shared/ui/settings-card/SettingsCard.stories.ts	shared	SettingsCard	2
entities/course/ui/CourseCard.stories.ts	entities	CourseCard	2
entities/course/ui/StudentCourseCard.stories.ts	entities	StudentCourseCard	2
features/toggle-theme/ui/ThemeToggler.stories.ts	features	ThemeToggler	3
features/change-language/ui/LanguageSwitcher.stories.ts	features	LanguageSwitcher	1
features/auth-by-email/ui/AuthByEmailForm.stories.ts	features	AuthByEmailForm	2
features/filter-users-by-role/ui/FilterUsersByRole.stories.ts	features	FilterUsersByRole	3

features/search-user/ui/SearchUser.stories.ts	features	SearchUser	2
features/add-user/ui/AddUserButton.stories.ts	features	AddUserButton	1
features/submit-assessment/ui/SubmitAssessmentButton.stories.ts	features	SubmitAssessmentBtn	2
widgets/profile-info-card/ui/ProfileInfoCard.stories.ts	widgets	ProfileInfoCard	3
widgets/security-settings-card/ui/SecuritySettingsCard.stories.ts	widgets	SecuritySettingsCard	2
widgets/admin-sidebar/ui/AdminSidebar.stories.ts	widgets	AdminSidebar	1
widgets/admin-header/ui/AdminHeader.stories.ts	widgets	AdminHeader	1
widgets/student-part-header/ui/StudentHeader.stories.ts	widgets	StudentHeader	1
widgets/teacher-part-header/ui/TeacherHeader.stories.ts	widgets	TeacherHeader	1

widgets/teacher-part-sidebar/ui/TeacherSidebar.stories.ts	widgets	TeacherSidebar	2
widgets/student-part-sidebar/ui/StudentSidebar.stories.ts	widgets	StudentSidebar	2
widgets/assessment-question-map/ui/QuestionMap.stories.ts	widgets	QuestionMap	2

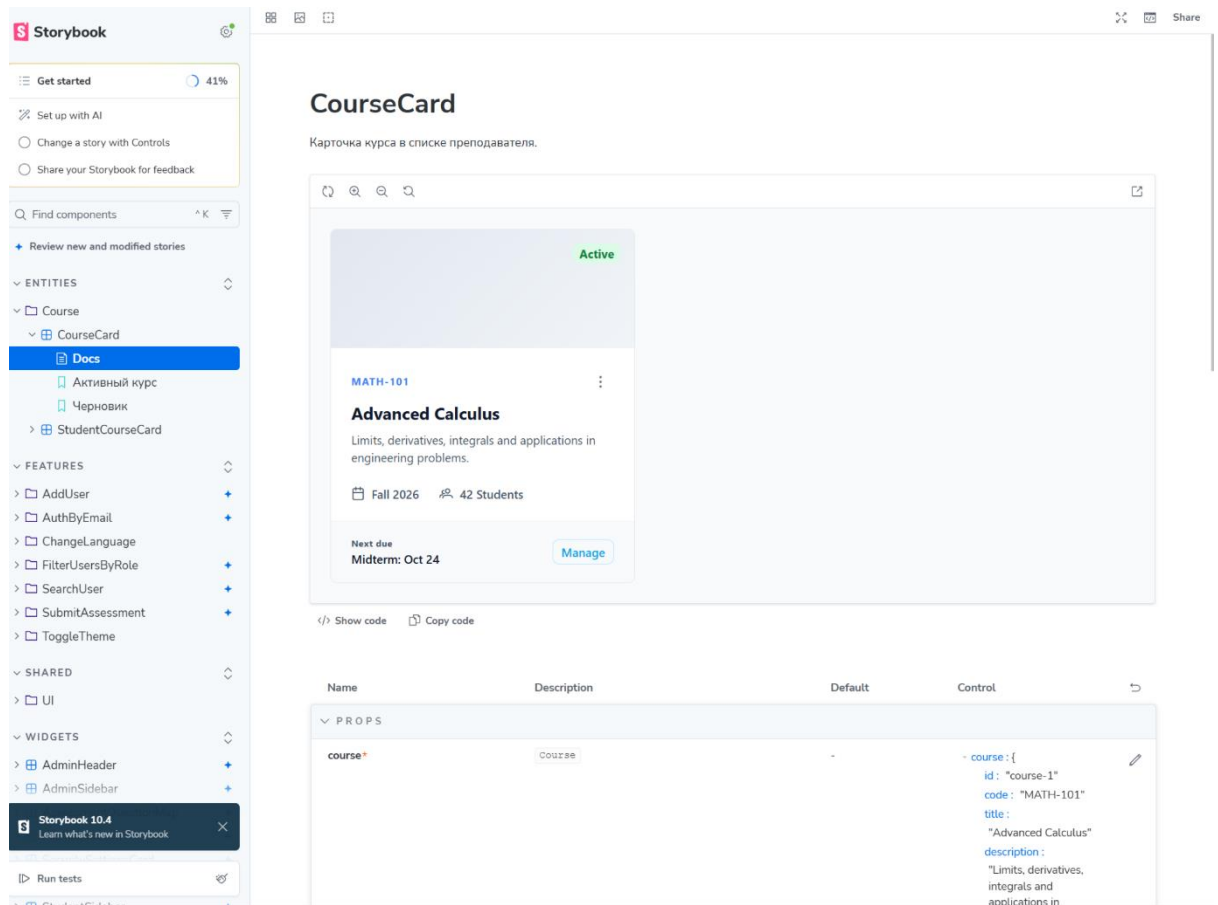


Рис. 37 интерфейс storybook-а

Каждый story-файл экспортирует объект meta с описанием компонента и именованные экспорты - сами stories. Свойство args определяет входные данные для конкретной story; в

Controls-панели Storybook каждый arg превращается в интерактивный элемент управления, позволяя изменять props прямо в браузере в реальном времени.

Тег `autodocs` в meta генерирует страницу «Docs» с автоматически извлечёнными props, типами и описаниями для каждого компонента.

5.3.4. Interaction tests и проверка доступности

Storybook поддерживает **interaction tests** - программные сценарии взаимодействия с компонентом в поле `play story`. Функция `play` выполняется автоматически после монтирования:

```
export const InteractionTest: Story = {
  name: 'Interaction: aria-label по теме',
  play: async ({ canvasElement }) => {
    const canvas = within(canvasElement)

    const button = canvas.getByRole('button')
    await expect(button).toBeVisible()

    const label = button.getAttribute('aria-label')
    await expect(['Тёмная тема', 'Светлая тема', 'Dark mode', 'Light mode']).toContain(label)
  }
}
```

Рис 38. Пример интерактивного теста

`getByRole('button')` находит элемент по ARIA-роли - это делает тест устойчивым к изменениям HTML-структуры компонента. Interaction tests запускаются тремя способами: в браузере Storybook через панель «Interactions», через Vitest + `@storybook/addon-vitest` (`npm run test:storybook`) и в CI как часть пайплайна.

Аддон `@storybook/addon-axe` интегрирует библиотеку `axe-core` для автоматической проверки доступности каждой открытой story по критериям WCAG 2.1 [31]. На момент написания работы для всех компонентов нарушения в категории «Violations» отсутствуют.

Заключение

В рамках выполнения работы был пройден полный цикл проектирования и разработки клиентской части образовательной платформы - от анализа предметной области до реализации функциональных модулей и организации автоматизированного контроля качества кода.

На этапе исследования предметной области была проведена формализация ключевых сущностей академической среды: иерархической организационной структуры вуза, ролевой модели участников учебного процесса, жизненного цикла курсов, заданий и тестовых сессий. Сравнительный анализ существующих LMS-платформ (Moodle, Canvas LMS, Stepik) позволил выявить наиболее эффективные архитектурные и интерфейсные практики, нашедшие отражение в проектных решениях работы.

В области архитектурного проектирования была изучена и применена на практике методология Feature-Sliced Design. Исследование показало, что строгая послойная декомпозиция кодовой базы (app, pages, widgets, features, entities, shared) и принцип однонаправленных зависимостей позволяют эффективно управлять сложностью крупного фронтенд-приложения, обеспечивая высокую степень независимости отдельных модулей и предсказуемость последствий любых изменений.

В ходе практической реализации были разработаны три изолированных пользовательских портала - административный, преподавательский и студенческий. Среди реализованных функциональных модулей: система аутентификации и авторизации на основе JWT с механизмом автоматического обновления токенов и ролевой защитой маршрутов; интерфейс управления пользователями с поддержкой массового импорта из Excel; визуальный редактор иерархической структуры университета; конструктор учебных курсов с управлением содержанием; интерфейс банка вопросов и конструктор тестов; студенческий портал прохождения тестирования.

В рамках обеспечения качества была выстроена многоуровневая система автоматических проверок: цепочка статического анализа (ESLint, Prettier), интегрированная в Git-хуки через Husky и lint-staged; набор из модульных и интеграционных тестов на Vitest, покрывающих функции-трансформации DTO, Zod-схемы валидации, сервисы и компоненты; система из story-файлов Storybook с историями для документирования компонентного API всех трех порталов.

Выполнение данной работы позволило на практике применить теоретические знания в области программной инженерии, освоить современный технологический стек фронтенд-

разработки и получить опыт принятия архитектурных решений в условиях реального проекта достаточной сложности. Полученные навыки работы с Vue 3, TypeScript, методологией Feature-Sliced Design, системами управления состоянием и инструментами обеспечения качества представляют практическую ценность для дальнейшей профессиональной деятельности в области разработки программного обеспечения.

Список литературы

1. Maslov, I. Exploring user experience of learning management system / I. Maslov, Sh. Nikou, P. Hansen // International Journal of Information and Learning Technology. – 2021. – Vol. 38, No. 4. – P. 344-363. – DOI 10.1108/ijilt-03-2021-0046. – EDN PTJKE.
2. Masveta, D., & Manyangara, M. E. (2025). The UX-UI continuum: exploring the interplay between user experience and user interface in e-learning platforms. Cogent Education, 12(1). <https://doi.org/10.1080/2331186X.2025.2536531>.
3. Амелин, П. П. Современные одностраничные веб-приложения / П. П. Амелин // Цифровые инструменты обеспечения устойчивого развития экономики и образования: новые подходы и актуальные проблемы : Сборник научных трудов III-й Национальной научно-практической конференции (с международным участием). В 2-х томах, Орел, 01 апреля 2024 года. – Орел: Российская академия народного хозяйства и государственной службы при Президенте РФ, 2024. – С. 209-213. – EDN TFTOMJ.
4. Лапенков, М. В. Одностраничные приложения. Их преимущества и недостатки / М. В. Лапенков // Инновации. Наука. Образование. – 2021. – № 27. – С. 867-872. – EDN ZAFKCP.
5. Бекирова, Э. А. Сравнительный анализ производительности и архитектурных подходов в javascript фреймворках: Vue.js и React / Э. А. Бекирова // Информационно-компьютерные технологии в экономике, образовании и социальной сфере. – 2024. – № 3(45). – С. 18-23. – EDN CIRVQA.
6. Волошко, М. Ю. Преимущество frontend-фреймворка vue.js перед другими frontend фреймворками / М. Ю. Волошко, А. Д. Соломыков, Н. Н. Куликова // Фундаментальные и прикладные научные исследования: актуальные вопросы, достижения и инновации : сборник статей XXXVI Международной научно-практической конференции : в 2 ч., Пенза, 27 июля 2020 года. Том Часть 1. – Пенза: "Наука и Просвещение" (ИП Гуляев Г.Ю.), 2020. – С. 187-189. – EDN SRXGQC.
7. Скоморохов, М. И. Feature-Sliced Design как универсальная архитектура для Frontend-проектов / М. И. Скоморохов // Международный студенческий научный вестник. – 2023. – № 6. – С. 1. – DOI 10.17513/msnv.21341. – EDN QTOHRM.

8. Васильев, Т. И. Реализация гибкой системы управления доступом в приложении / Т. И. Васильев // Труды Института системного программирования РАН. – 2025. – Т. 37, № 4-1. – С. 147-160. – DOI 10.15514/ISPRAS-2025-37(4)-8. – EDN WDHJJK.
9. Ковтун, Д. П. Использование управления доступом на основе атрибутов и mTLS в микросервисной архитектуре / Д. П. Ковтун, О. Р. Лапоница // International Journal of Open Information Technologies. – 2025. – Т. 13, № 6. – С. 75-85. – EDN NCDSOC.
10. Макаров, К. С. Скриншотное тестирование как многоаспектный вид автоматизированной динамической верификации веб-приложений / К. С. Макаров, Р. И. Фаткин // Программные системы и вычислительные методы. – 2025. – № 1. – С. 32-54. – DOI 10.7256/2454-0714.2025.1.73535. – EDN UVGEBС.
11. Vue.js 3 : официальная документация [Электронный ресурс]. – Режим доступа: <https://vuejs.org/guide/introduction.html>. – Загл. с экрана.
12. Vue Router : официальная документация [Электронный ресурс]. – Режим доступа: <https://router.vuejs.org/guide/>. – Загл. с экрана.
13. Pinia : официальная документация [Электронный ресурс]. – Режим доступа: <https://pinia.vuejs.org/introduction.html>. – Загл. с экрана.
14. Vue I18n : официальная документация [Электронный ресурс]. – Режим доступа: <https://vue-i18n.intlify.dev/guide/introduction.html>. – Загл. с экрана.
15. PrimeVue : официальная документация UI-компонентов [Электронный ресурс]. – Режим доступа: <https://primevue.org/introduction/>. – Загл. с экрана.
16. TanStack Query for Vue : официальная документация [Электронный ресурс]. – Режим доступа: <https://tanstack.com/query/latest/docs/framework/vue/overview>. – Загл. с экрана.
17. VeeValidate : официальная документация библиотеки валидации форм [Электронный ресурс]. – Режим доступа: <https://vee-validate.logaretm.com/v4/guide/overview/>. – Загл. с экрана.
18. Zod : официальная документация библиотеки схем и валидации [Электронный ресурс]. – Режим доступа: <https://zod.dev/>. – Загл. с экрана.
19. Vite : официальная документация сборщика [Электронный ресурс]. – Режим доступа: <https://vitejs.dev/guide/>. – Загл. с экрана.
20. TypeScript : официальная документация языка программирования [Электронный ресурс]. – Режим доступа: <https://www.typescriptlang.org/docs/>. – Загл. с экрана.

21. Vitest : официальная документация тест-раннера [Электронный ресурс]. – Режим доступа: <https://vitest.dev/guide/>. – Загл. с экрана.
22. Vue Test Utils : официальная документация утилит тестирования компонентов [Электронный ресурс]. – Режим доступа: <https://test-utils.vuejs.org/introduction/>. – Загл. с экрана.
23. Storybook : официальная документация среды разработки компонентов [Электронный ресурс]. – Режим доступа: <https://storybook.js.org/docs>. – Загл. с экрана.
24. ESLint : официальная документация линтера [Электронный ресурс]. – Режим доступа: <https://eslint.org/docs/latest/>. – Загл. с экрана.
25. Prettier : официальная документация форматтера кода [Электронный ресурс]. – Режим доступа: <https://prettier.io/docs/>. – Загл. с экрана.
26. Husky : официальная документация инструмента Git-хуков [Электронный ресурс]. – Режим доступа: <https://typicode.github.io/husky/>. – Загл. с экрана.
27. Feature-Sliced Design : официальная документация методологии проектирования фронтенд-приложений [Электронный ресурс]. – Режим доступа: <https://feature-sliced.design/docs/get-started/overview>. – Загл. с экрана.
28. Docker : официальная документация платформы контейнеризации [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/get-started/overview/>. – Загл. с экрана.
29. Nginx : официальная документация веб-сервера [Электронный ресурс]. – Режим доступа: <https://nginx.org/en/docs/>. – Загл. с экрана.
30. RFC 7519 : JSON Web Token (JWT) / М. Jones, J. Bradley, N. Sakimura // Internet Engineering Task Force (IETF). – 2015. – Режим доступа: <https://datatracker.ietf.org/doc/html/rfc7519>. – Загл. с экрана.
31. WCAG 2.1 : Web Content Accessibility Guidelines // W3C Recommendation. – 2018. – Режим доступа: <https://www.w3.org/TR/WCAG21/>. – Загл. с экрана.
32. Макконнелл, С. Совершенный код: практическое руководство по разработке программного обеспечения / С. Макконнелл. - 2-е изд. - Москва: Вильямс, 2019. - 896 с. - ISBN 978-5-8459-2110-8. - Текст: непосредственный.